



Hantro VC9000D Video Decoder

API User Manual

Document Revision 1.03

3 January 2024

This document is compatible with the following product:

Hantro VC9000D software release version 2.4.80

VERISILICON

LEVEL A: PROPRIETARY INFORMATION

Legal Notices

COPYRIGHT INFORMATION

This document contains proprietary information of VeriSilicon Holdings Co., Ltd. VeriSilicon reserves the right to make changes to any products herein at any time without notice. VeriSilicon does not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by VeriSilicon; nor does the purchase or use of a product from VeriSilicon convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of VeriSilicon or third parties.

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained in this VeriSilicon proprietary copyright document may be re-distributed by VeriSilicon customer SoC vendors who have licensed the related Hantro IP, with the understanding that the material is re-packaged/re-branded as a licensee (SoC vendor) written/labeled document. Those portions of the SoC vendor document adapted from the VeriSilicon Hantro material should contain a notice similar to: "This section contains copyright material disclosed with permission of VeriSilicon Holdings Co., Ltd."

(VeriSilicon Distribution **LEVEL A: PROPRIETARY INFORMATION**)

TRADEMARK ACKNOWLEDGMENT

VeriSilicon, the VeriSilicon logo, Hantro and the Hantro logo are the trademarks of VeriSilicon Holdings Co., Ltd. in the United States and/or other jurisdictions. All other trademarks are the property of their respective holders.

For our current distributors, sales offices, design resource centers, and product information, visit our web site located at <http://www.verisilicon.com>.

VeriSilicon Confidential, Copyright © 2024 by VeriSilicon Holdings Co., Ltd. All rights reserved worldwide.

Preface

This chapter introduces the *Hantro VC9000D Video Decoder API User Manual*.

About This Document

This document introduces the application programming interface (API) of Hantro VC9000D video decoder IP. It describes the data types, enumerations, structures, and functions in the API, which is written in C code. It also provides a programming overview and examples for your reference.

Additional Reading

- Hantro VC9000D Video Decoder Hardware Features
- Hantro VC9000D Video Decoder Memory Buffer and Format Organization
- Hantro VC9000D Video Decoder Test Bench User Manual
- Hantro VC9000D Video Decoder Wrapper Layer API User Manual

Table of Contents

Legal Notices	1
Preface	2
1 Overview	8
1.1 Supported Standards	8
2 Decoder Programming Overview	9
2.1 Usage of the Decoder	9
2.2 Initialization of the Decoder	11
2.2.1 Support for Multiple Streams.	12
2.3 Getting Information About the Stream Parameters	12
2.4 Setting Information According to the Parameters.	13
2.5 Allocating External Buffers	13
2.6 Decoding a Stream	13
2.7 End of Stream	15
2.8 Decoder Shutdown	16
2.9 Low Latency Decoding	16
2.10 Error Concealment Policy	18
3 Decoder Application Examples	19
4 Module Documentation	23
4.1 COMMON.	23
4.1.1 Marco Definition	26
4.1.1.1 IS_RGB_4PLANE	26
4.1.1.2 IS_PIC_PACKED_RGB	27
4.1.1.3 IS_PIC_PLANAR_RGB	27
4.1.1.4 IS_PIC_RGB	27
4.1.1.5 IS_PIC_ALPHA_FORMAT	27

4.1.1.6	IS_PIC_XRGB_FORMAT	28
4.1.1.7	IS_PIC_MONOCHROME	28
4.1.1.8	IS_PIC_YCbCr420	29
4.1.1.9	IS_PIC_YCbCr422	29
4.1.1.10	IS_PIC_YCbCr444	30
4.1.1.11	IS_PIC_PLANAR	30
4.1.1.12	IS_PIC_FIELD_PLANAR	30
4.1.1.13	IS_PIC_1010	31
4.1.1.14	IS_PIC_NV21	31
4.1.1.15	IS_PIC_SEMIPLANAR	32
4.1.1.16	IS_PIC_8BIT	32
4.1.1.17	IS_PIC_10BIT	32
4.1.1.18	IS_PIC_12BIT	33
4.1.1.19	IS_PIC_16BIT	33
4.1.1.20	IS_PIC_24BIT	33
4.1.1.21	IS_PIC_32BIT	34
4.1.1.22	IS_PIC_48BIT	34
4.1.1.23	IS_PIC_TILE	34
4.1.1.24	IS_PIC_RFC	34
4.1.1.25	IS_PIC_DEC400	35
4.1.1.26	IS_PIC_TILED4x4	35
4.1.1.27	IS_PIC_REF_TILED8x8	36
4.1.1.28	IS_PIC_TILED8x8	36
4.1.1.29	IS_PIC_TILED16x16	36
4.1.1.30	IS_PIC_TILED16x16_COMP	36
4.1.1.31	IS_PIC_TILED32x8_COMP	37
4.1.1.32	IS_PIC_422PACKED	37
4.1.1.33	IS_PIC_TILED128x2	37
4.1.1.34	IS_PIC_RGB_TILED64x64	37
4.1.1.35	IS_PP_IN_P010	38
4.1.1.36	IS_PP_IN_TILED4x4	38
4.1.1.37	IS_PP_IN_RGB	38
4.1.1.38	IS_PP_IN_8BIT	39
4.1.1.39	IS_PP_IN_10BIT	39
4.1.1.40	IS_PP_IN_RFC	39
4.1.1.41	TRANSE32	39
4.1.2	Enumeration Type	40
4.1.2.1	DecDpbMode	40

4.1.2.2	DecDecoderMode	40
4.1.2.3	DecAuxInfo	41
4.1.2.4	DelogoMode	41
4.1.2.5	DecPictureFormat	42
4.1.2.6	DecPicCodingType	49
4.1.2.7	DecErrorHandling	50
4.1.2.8	DecPicAlignment	51
4.1.2.9	DecCodec	51
4.1.2.10	DecRet	52
4.1.2.11	DecDARFormat	54
4.1.2.12	DecInputFormat	55
4.1.2.13	DecSkipFrameMode	55
4.1.2.14	DecVideoRange	56
4.1.2.15	DecColorSpace	56
4.1.2.16	DecBufferType	57
4.1.2.17	DecUserDataTypes	57
4.1.3	Functions	58
4.1.3.1	VCDecGetAPIVersion()	58
4.1.3.2	VCDecGetBuild()	58
4.1.3.3	VCDecMCGetCoreCount()	59
4.1.3.4	VCDecUnpackMetaData()	59
4.1.3.5	VCDecInit()	60
4.1.3.6	VCDecGetInfo()	60
4.1.3.7	VCDecSetInfo()	61
4.1.3.8	VCDecDecode()	61
4.1.3.9	VCDecNextPicture()	63
4.1.3.10	VCDecPictureConsumed()	64
4.1.3.11	VCDecEndOfStream()	64
4.1.3.12	VCDecGetBufferInfo()	65
4.1.3.13	VCDecAddBuffer()	65
4.1.3.14	VCDecUseExtraFrmBuffers()	66
4.1.3.15	VCDecPeek()	66
4.1.3.16	VCDecAbort()	67
4.1.3.17	VCDecAbortAfter()	67
4.1.3.18	VCDecSetNoReorder()	68
4.1.3.19	VCDecRelease()	68
4.1.3.20	VCDecUpdateStrmInfoCtrl()	69
4.1.3.21	VCDecGetUserData()	69

4.1.3.22 VCDecSetCustomInfo()	70
4.1.3.23 VCDecRetStr()	70
5 Data Structure Documentation	72
5.1 DecApiVersion	72
5.2 DecBufferInfo	72
5.2.1 Field Documentation	73
5.2.1.1 next_buf_size	73
5.3 DecConfig	73
5.4 DecCropParams	74
5.5 DecInitConfig	75
5.5.1 Field Documentation	77
5.5.1.1 multicore_poll_period	77
5.6 DecInputParameters	77
5.7 DecMCConfig	79
5.8 DecMetaData	79
5.9 DecOutput	80
5.9.1 Field Documentation	81
5.9.1.1 llstrm_curr_address	81
5.10 DecParams	81
5.11 DecPicture	82
5.12 DecPictureInfo	83
5.13 DecPictures	85
5.14 DecSequenceInfo	85
5.15 DecSliceInfo	88
5.16 DecSwHwBuild	88
5.17 DecTime	89
5.18 DecUserConf	89
5.19 DelogoConfig	90
5.20 ErrorParams	91
5.21 PpUnitConfig	92
5.21.1 Field Documentation	96
5.21.1.1 enabled	96
5.21.1.2 width	96
5.21.1.3 height	96
5.22 SEI_buffer	96
Document Revision History	98

List of Figures

2.1 Decoder Initialization	10
2.2 Simple Decoding Process	11
2.3 Call and Message Sequence Chart for Decoding	14
2.4 Low Latency Decoding	17

VeriSilicon Distribution Level A:
Proprietary Information

1 Overview

This document describes the Application Programming Interface (API) of the Hantro VC9000D hardware based decoder hardware.

Details of the Hantro Video Decoder API are described in this document.

- First, components such as data types, return codes, enumerations and relevant structures are described.
- Then, function syntax and description is presented.
- Finally, a programming overview and examples are presented.

The code is written in C and parameter types follow standard C conventions. This document assumes that the reader understands the fundamentals of the C-language and the VCDEC standards.

Currently, there are no deprecated functions in this API.

1.1 Supported Standards

The API discussed in this version of the document is compatible with the following video coding standards and profiles. Additional levels may be supported via software.

2 Decoder Programming Overview

This section describes the decoder programming.

Note: xxx means a specific format, including Hvc, H264, Avs2, Avs, Av1, Vp6, Vp8, Vp9, Jpeg, Mpeg2, Mpeg4, Vc1, Rv. For example xxxDeclnit can be HvcDeclnit ,H264Declnit ,etc.

- Usage of the Decoder
- Initialization of the Decoder
- Getting Information About the Stream Parameters
- Setting Information According to the Stream Parameters
- Allocating External Buffers
- Decoding a Stream
- End of Stream
- Decoder Shutdown
- Low Latency Decoding
- Error Concealment Policy

2.1 Usage of the Decoder

The decoder is operated via the Application Programming Interface (API). we present an example of HEVC stream decoding in the figures below.

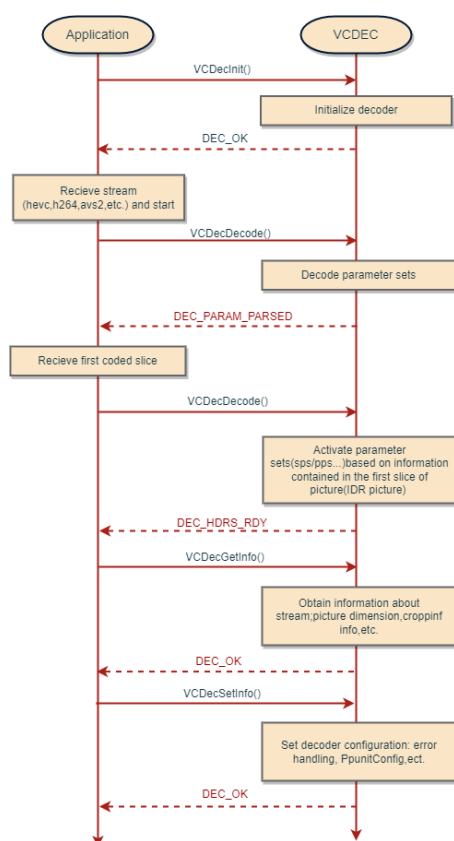


Figure 2.1 Decoder Initialization

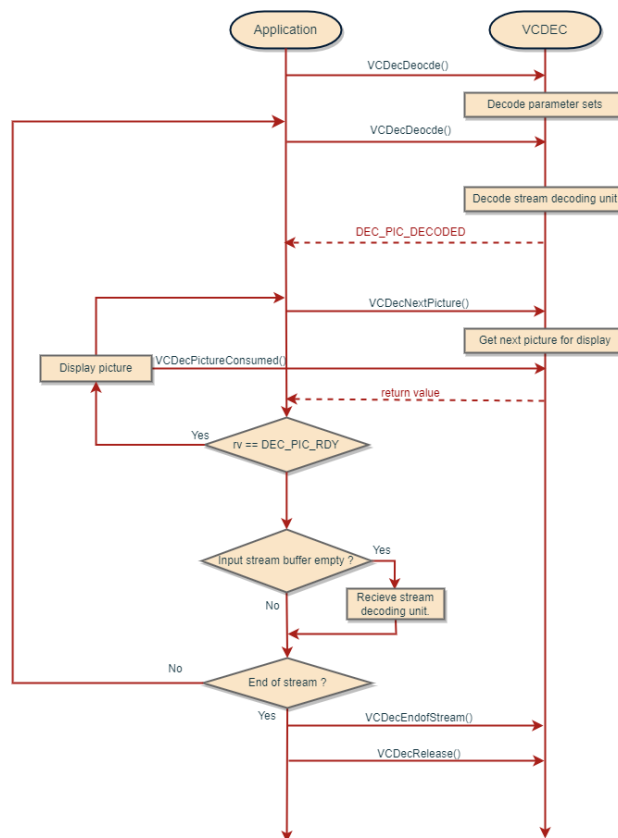


Figure 2.2 Simple Decoding Process

2.2 Initialization of the Decoder

The decoding procedure is started with the VCDecInit function, which reserves memory for the decoder instance and initializes the decoder pointer according to a specific format. So it actually calls xxxDecInit function. For example, for HEVC format, VCDecInit function actually calls HvcDecInit to initialize the instance data.

If the initialization is successful, the call returns DEC_OK and stores the decoder instance in the place pointed to by inst. This decoder instance will be used as a parameter in all subsequent decoder API function calls. It is possible to initialize multiple decoder instances.

The decoder still needs to decode and activate some of the parameter sets before it will be fully initialized (e.g. required resources will be allocated only after the size of the picture and number of reference pictures have been decoded from the stream). When the necessary buffers are allocated inside, the decoder has allocated all its resources when VCDecDecode has returned DEC_HDRS_RDY.

Note: Memory requirements are specified in the VC9000D HW Integration Guide.

Some stream parameters may change during the decoding and in that case, the decoder will need to reinitialize itself. Any such change in the parameters is signaled by returning the same DEC_HDRS_RDY.

- **Support for Multiple Streams**

2.2.1 Support for Multiple Streams

The decoder product has support for multiple software instances sharing single decoder hardware. This means that it is possible to initialize several decoder instances and use them for decoding different streams. The only limitation is that the hardware can only decode one frame at a time so the decoder instances share the hardware with a time slicing principle.

There are two methods the application can use to manage multiple streams:

1. The application maintains and manages multiple decoder instances. After one frame is decoded, the application can switch to another frame in either the same instance or a different instance.
2. The application utilizes multiple processes, with each process managing one stream. The decoder driver incorporates hardware resource protection to avoid conflicts between the different processes. After each frame is decoded, the hardware resource is released for other processes to use.

2.3 Getting Information About the Stream Parameters

The VCDecGetInfo function actually calls xxxDecGetInfo according to a specific format to read information from the decoder and give access to the data in the activated parameter sets. The successful activation of the parameter set is signaled by a DEC_HDRS_RDY return in a call from the VCDecDecode function. Picture size, cropping information and video range are some of the parameters that can be retrieved from the decoder.

When the activated parameters are not supported by the decoder, VCDecDecode will return

DEC_STREAM_NOT_SUPPORTED. Decoding cannot be performed on this stream.

Picture sizes that are not multiples of 8 pixels (minimum CU size) are supported although the size of the decoded picture is always a multiple of 8 pixels. These picture sizes are indicated in the stream by using cropping parameters in sequence parameter sets. The cropping parameters are available through the VCDecGetInfo function.

NOTE: The decoder does not perform the required cropping; this has to be done externally.

2.4 Setting Information According to the Parameters.

After getting stream parameters by calling VCDecGetInfo, it needs to use these information to initialize some parameters, such as the configuration of post processor(crop parameters, scale parameters, output format, etc.), the error policy for SW and so on.

2.5 Allocating External Buffers

The decoder can support allocating necessary external buffers. When the VCDecDecode function returns DEC_WAITING_FOR_BUFFER, then the function VCDecGetBufferInfo(It actually calls xxxDecGetBufferInfo function) is used for reading information about the external buffer, such as the size and the expected number of the external buffer. The client then allocates the required external buffers and calls the function VCDecAddBuffer(it actually calls xxxDecAddBuffer) to add those buffers to the decoder. After adding the minimum number of external buffers, the system can decode again and can add extra external buffers to the decoder going forward.

NOTE: After VCDecDecode returns DEC_HDRS_RDY, if new external buffers are needed in the case where the resolution in the new header is larger than the previously allocated output buffer or the DPB size is changed, the client is responsible for freeing external buffers.

2.6 Decoding a Stream

The entire stream decoding work is done by the VCDecDecode function. This function actually calls xxxDecDecode function according to a specific format. A call and message sequence

chart of a decoder usage is depicted in the figure below.

NOTE: The start of the stream buffer must be either 128-bit aligned or accessible for reading from the previous 128-bit aligned position.

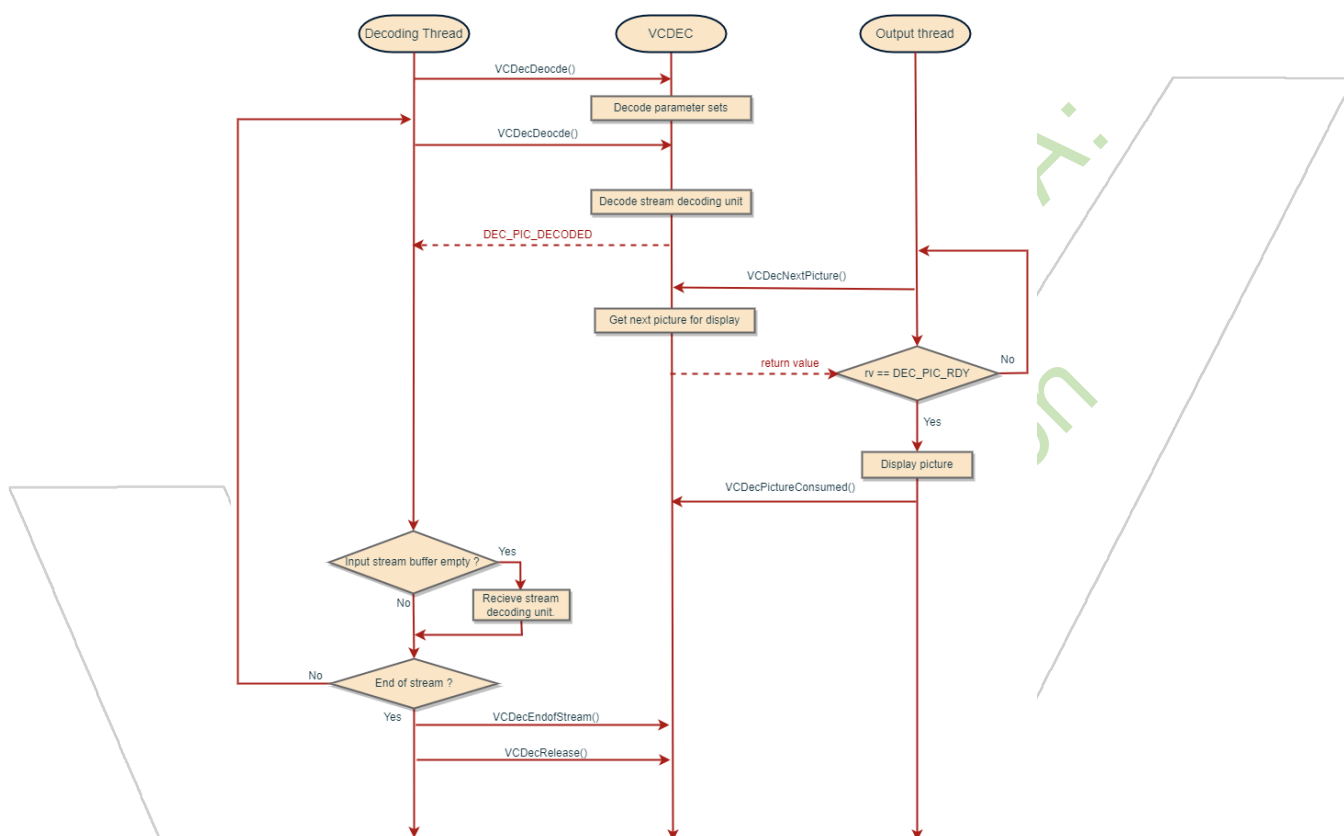


Figure 2.3 Call and Message Sequence Chart for Decoding

At the beginning of a new stream the decoder first decodes the sequence and picture parameter sets.. If the parameter sets are successfully decoded and the first slice in the stream activates parameter sets supported by the decoder, VCDecDecode will return DEC_HDRS_RDY. When a non-supported parameter set is encountered, the decoder returns DEC_STREAM_NOT_SUPPORTED and the decoding has to stop.

After a picture is decoded by the hardware, VCDecDecode will return DEC_PIC_DECODED. At this point the application should check if there are any pictures in the picture buffer of the decoder that have to be displayed. It should be remembered that the decoding order of the pictures may be different from the display order of the pictures depending on the configuration during VCDecInit. Therefore, the picture that was just finished may not be the one to be displayed next.

It may happen that after the decoding of a picture is finished, multiple pictures have to be

displayed before decoding the next one due to the restrictions on picture buffering. To get all available pictures to be displayed, the application has to call `VCDecNextPicture` function repeatedly until the return value indicates that there are no more pictures to be displayed.

The output structure of `VCDecNextPicture` function contains a pointer to the next picture to be displayed. The output structure also contains the identifier of the picture (the one given by the user when the picture was decoded), average decoding time in cycles per macroblock, and an indication whether the picture is an Instantaneous Decoder Refresh (IDR) picture or not.

The decoder can handle erroneous streams. When an error is detected the decoder will try to conceal the effects of the corrupted data. The application can initialize the decoder in `video freeze` error concealment mode, where the video will be frozen after an error is detected. The video will be frozen until the next correct key frame (IDR) is decoded.

If the reordering of pictures is disabled when `VCDecInit` is called, `VCDecNextPicture` will always return the last decoded picture. When the decoder buffers pictures for reordering purposes and the entire stream data has been decoded, the picture buffer has to be flushed. For flushing all the pictures from the DPB, the function `VCDecNextPicture` will be called until there are no pictures to be output.

The output picture of the decoder is in semi-planar YCbCr 4:2:0 format or tiled 4x4 format based on the value of config in `VCDecInit`.

NOTE: The picture buffers must be treated as a read-only area as it might be used as references in decoding of the next pictures.

When down scaling is enabled in `VCDecInit`, the down scaled buffer may also be output along with the display buffer through `VCDecNextPicture`.

2.7 End of Stream

When there is no more data in the input buffer, which means the last frame of this stream has been sent to the decoder, `VCDecEndofStream` function will be called. This function will wait for decoding done and flush remaining pictures in DPB into output fifo. Then it will wake up output thread to output remaining pictures.

2.8 Decoder Shutdown

The decoder shutdown is performed by the `VCDecRelease` function (It actually calls `xxxDecRelease` function). The purpose of this call is to close a decoder instance and release all allocated resources related to that instance. This function must not be called twice for the same decoder instance.

2.9 Low Latency Decoding

The purpose of this design is to reduce the latency which exists of the stream saved into SDRAM and HW decoding start. Before low latency designed, the minimum decoding unit in HW is one slice. But in low latency mode, the minimum decoding unit in HW is 256bytes in one slice, in this case the latency which exists of the stream saved into SDRAM and HW decoding start is the storage time of 256bytes.

There are two methods to send stream data to HW if low latency mode enabled, including frame mode and stream mode.

1. In frame mode, each packet will be sent to decoder continuously with a time interval until the end of one frame. (each packet does not contain frame boundary). If you know each frame boundary's location in the stream, they can take frame mode for consideration.

2. In stream mode, each packet will be sent to decoder continuously with a time interval until the end of stream (some packets may contain frame boundary). If you do not know each frame boundary's location in the stream, they can take stream mode for reference.

Both of these two methods will reduce the complexity of stream sending thread because stream sending thread no need to compute the start base address of stream for each frame. We present an decoding flow with low latency feature in the figures below.

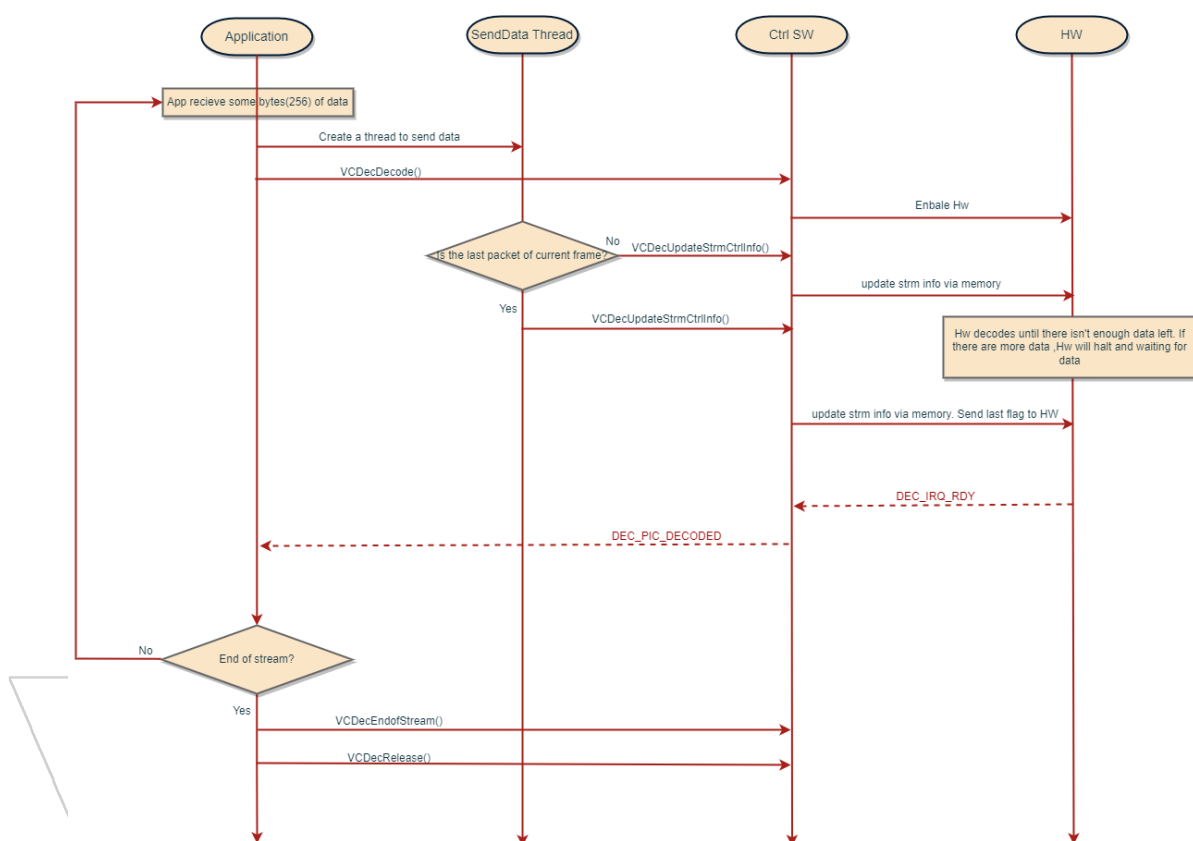


Figure 2.4 Low Latency Decoding

The functions used in the examples are blowed.

```

// send_bytestrm_task is a send thread in low latency mode.
void* send_bytestrm_task(void* param) {
    while(!frame_start);
    while(!process_end_flag){
        while(send_len <= fream_len || pic_decoded == 0) {
            if(error)
                break;
            update stream_info;
            VCDecUpdateStrmInfoCtrl;
            update stream_len;
        }
        if(pic_decoded) {
            pic_decoded = 0;
            frame_end = 1;
            while(!frame_start);
        }
    }
}

// UpdateStrmInfoCtrl updates the stream_info struct called inside the api and updates the information for
// the hardware after starting decoding
UpdateStrmInfoCtrl(inst, stream_info) {
    update dec_cont->stream_info;
    if(update_reg_flag){
        while(The hardware is not booting);
    }
}
  
```

```
update stream info for HW;
if(frame decoding finished){
    update_reg_flag = 0;
}
}
```

2.10 Error Concealment Policy

There are three typical software error concealment policies. User can choose an appropriate policy base on your situation.

- Focus on the clarity of the video, just display the correct pictures.
- Focus on the fluency of the video, display all pictures including error pictures.
- Both clarity and fluency of the video are considered, user can set a tolerable error ratio by parameters. So our decoder will display the pictures whose error ratio are less than or equal to the ratio user set.

3 Decoder Application Examples

The purpose of the following examples is to show how the decoding of a stream can be done. The functions used in the examples are from the VCDEC API, except for the functions `SetClientByTBCfg`, `FindExtBufferIndex`, `SetupDefaultParams`, `ReleaseExtBuffers`, `HeadersDecoded` and `DWLInit` that are system/application dependent.

Note: The examples are simplified and there is no error handling. The program code given in the examples below should be considered as pseudo code and is not guaranteed to compile as such.

```
#include "basetype.h"
#include "dwl.h"
#include "command_line_parser.h"
#include "dwlthread.h"
#include "demuxer_types.h"
#include "bytestream_parser.h"
static void SetClientByTBCfg(struct Client* client)
int main(void) {
    struct Client client;
    struct TestParams test_params;
    struct DWLLinearMem stream_mem;
    int ret;
    struct DecSwHwBuild dec_build;
    struct DWLLinearMem *buffer = NULL;
    struct DWLInitParam dwl_params = {DWL_CLIENT_TYPE_HEVC_DEC};
    struct DecOutput dec_output;
    u32 pic_decode_number = 1;
    pthread_t output_thread = 0;
    struct DecSequenceInfo dec_info={0};
    struct DecBufferInfo buf_info={0};
    SetupDefaultParams(&test_params);
    dec_build = VCDecGetBuild();
    dwl = DWLInit(&dwl_params);
    SetClientByTBCfg(&client);
    rv = VCDecInit((const void*)&dec_inst, &init_config);
    if(rv != DEC_OK)
        goto end;
    do{
        DWLmemset(&input_param, 0, sizeof(struct DecInputParameters));
        ret = VCDecDecode(dec_inst, &dec_output, &input_param);
        switch (ret) {
            case DEC_STREAM_NOT_SUPPORTED: {
                goto end;
            }
            case DEC_HDRS_RDY: {
                //hdrs_rdy = 1;
                // Stream headers were successfully decoded-> stream information is available for query now
                rv = VCDecGetInfo(dec_inst, &dec_info);
            }
        }
    } while (ret != DEC_OK);
    goto end;
}
```

```

    if (rv != DEC_OK)
        goto end;
    memcpy(config.ppu_cfg, test_params.ppu_cfg, sizeof(config.ppu_cfg));
    rv = VCDecSetInfo(dec_inst, &config);
    if (rv != DEC_OK)
        goto end;
    HeadersDecoded(&test_params, demuxer, yuvsink, &dec_info);
    if (user_buffer_mode == USER_SPECIFIED_BUFFER) {
        struct DWLLinearMem mem;
        u32 buffer_size;
        // Allocate external buffers, which will be used when calling VCDecDecode.
        rv = VCDecGetBufferInfo(dec_inst, &buf_info);
        buffer_size = buf_info.next_buf_size;
        for (int i=0; i < buf_info.buf_num; i++) {
            mem.mem_type = DWL_MEM_TYPE_DPB;
            DWLMallocLinear(dw1, buffer_size, &mem);
            DWLmemset(mem.virtual_address, 0, mem.size);
            client.ext_buffers[client.num_buffers] = mem;
            client.buffer_consumed[client.num_buffers] = 1;
            client.num_buffers++;
        }
    }
    break;
}

case DEC_ADVANCED_TOOLS: {
    // ASO/STREAM ERROR was noticed in the stream. The decoder has to reallocate resources.
    ASSERT(dec_output.data_left);
    //we should have some data left
    // Used to indicate that picture decoding needs to finalized prior to corrupting Next picture
    // Used to indicate that picture decoding needs to finalized prior to corrupting next picture pic_rdy
    = 0;
    break;
}

case DEC_PIC_DECODED: {
    // If enough pictures decoded -> force decoding to end by setting that no more stream is available.
    if (pic_decode_number == test_params.num_of_decoded_pics)
        dec_input.data_len = 0;
    // Increment decoding number for every decoded picture
    pic_decode_number++;
#ifdef OUTPUT_THREAD
    if (!client.output_thread_run) {
        client.output_thread_run = 1;
        pthread_create(&output_thread, NULL, OutputThread, &client);
    }
#else
    // Call OutputThread to output all the available display pictures.
    output_thread = 0; // for building warning.
    client.output_thread_run = 1;
    OutputThread(&client);
#endif
}

case DEC_PENDING_FLUSH: {
    // use function VCDecNextPicture() to obtain next picture in display order. Function is called until no
    // more images are ready for display .
    // Call OutputThread to output all the available display pictures.
    client.output_thread_run = 1;
    OutputThread(&client);
    break;
}

case DEC_WAITING_FOR_BUFFER: {
    struct DWLLinearMem mem;
    u32 buffer_size;
    rv = VCDecGetBufferInfo(dec_inst, &buf_info);
    if (user_buffer_mode == PRE_ADDED_BUFFER) {
        // Free external buffers when necessary.
        if (rv == DEC_WAITING_FOR_BUFFER && buf_info.buf_to_free.virtual_address) {
            int bufi = FindExtBufferIndex(buf_info.buf_to_free.virtual_address,
                                         &client);

```

```

        DWLFreeLinear(dw1, &client.ext_buffers[bufl]);
        DWLmemcpy((void *)&client.ext_buffers[bufl],
            (void *)&client.ext_buffers[bufl+1],
            (client.num_buffers - bufl - 1) * sizeof(client.ext_buffers[0]));
        client.num_buffers--;
    }
    if (buf_info.next_buf_size) {
        // Only add minimum required buffers at first.
        buffer_size = buf_info.next_buf_size;
        for (int i=0; i < buf_info.buf_num; i++) {
            DWLMallocLinear(dw1, buffer_size, &mem);
            DWLmemset(mem.virtual_address, 0, mem.size);
            rv = VCDecAddBuffer(dec_inst, &mem);
            client.ext_buffers[client.num_buffers] = mem;
            client.buffer_consumed[client.num_buffers] = 1;
            client.num_buffers++;
        }
    }
}
break;
case DEC_OK:
    // nothing to do, just call again.
case DEC_STRM_PROCESSED:
case DEC_DP_HDRS_RDY:
    // New stream headers decoded, data partitioned stream(mpeg4 specific)
case DEC_SLICE_RDY:
    // A slice was decoded(vp8/jpeg specifyc)
case DEC_NONREF_PIC_SKIPPED:
case DEC_SCAN_PROCESSED:
    // Scan Processed(pjpeg specific)
case DEC_ABORTED:
case DEC_FLUSHED:
case DEC_BUF_EMPTY:
case DEC_RESOLUTION_CHANGED:
case DEC_VOS_END:
case DEC_PARAM_SET_PARSED:
case DEC_SEI_PARSED:
case DEC_PB_PIC_SKIPPED:
case DEC_STRM_ERROR:
    // Used to indicate that picture decoding needs to finalized prior to corrupting next picture pic_rdy =
    0;
case DEC_HDRS_NOT_RDY:
case DEC_NO_REFERENCE:
case DEC_PARAM_SET_ERROR:
case DEC_SEI_ERROR:
case DEC_SLICE_HDR_ERROR:
case DEC_REF_PICS_ERROR:
case DEC_NALUNIT_ERROR:
case DEC_AU_BOUNDARY_ERROR:
case DEC_STREAM_ERROR_DETECTED:
    break;
case DEC_NO_DECODING_BUFFER:
    break;
case DEC_HW_TIMEOUT:
    goto end;
default: {
    // fatal error: need shutdown vcd.
    if (ret < 0) {
        goto end;
    }
}
}
} while (dec_input.data_len > 0);
end:
if (output_thread) pthread_join(output_thread, NULL);
if (dec_inst) VCDecEndOfStream(dec_inst);
ReleaseExtBuffers(dw1, &client);

```

```

    if (dec_inst) VCDecRelease(dec_inst);
    if (dwl) DWLRelease(dwl)
}
// Release external buffer
void ReleaseExtBuffers(const void *dwl, struct Client *client) {
    u32 num_buffers = client->num_buffers;
    struct DWLLinearMem *ext_buffers = client->ext_buffers;
    if (!dwl) return;
    for (int i=0; i<num_buffers; i++) {
        while (client->buffer_consumed[i] == 0) sched_yield();
        if (ext_buffers[i].virtual_address != NULL) {
            DWLFreeLinear(dwl, &ext_buffers[i]);
            DWLmemset(&ext_buffers[i], 0, sizeof(ext_buffers[i]));
        }
    }
}
void *OutputThread(void *arg) {
    while (output_thread_run) {
        ret = VCDecNextPicture(dec_inst, &dec_picture);
        if (ret == DEC_PIC_RDY) {
            id = FindExtBufferIndex(dec_picture.pictures[0].luma.virtual_address, client);
            client->buffer_consumed[id] = 0;
            fflush(stdout);
            PictureReady(client, &dec_picture);
            for (i = 0; i < DEC_MAX_OUT_COUNT; i++) {
                if (dec_picture.pictures[i].luma.virtual_address == NULL)
                    continue;
            }
            VCDecPictureConsumed(dec_inst, &dec_picture);
            client->buffer_consumed[id] = 1;
            pic_display_number++;
        } else if (ret == DEC_END_OF_STREAM) {
            break;
        }
    }
    return NULL;
}
u32 FindExtBufferIndex(u32 *addr, struct Client *client) {
    u32 i;
    u32 num_buffers = client->num_buffers;
    struct DWLLinearMem *ext_buffers = client->ext_buffers;
    for (i = 0; i < num_buffers; i++) {
        if (ext_buffers[i].virtual_address == addr)
            break;
    }
    ASSERT(i < num_buffers);
    return i;
}

```

4 Module Documentation

4.1 COMMON

Marco Definition

- **#define MAX_ASIC_CORES 4**

Maximum number of cores supported in multi-core configuration. For G2, multi-core currently not supported.

- **#define MAX_REG_COUNT 768**

- **#define BT601 0**

Different standards determining the coeff used to convert from YUV domain to RGB domain. Suffix "_L" means limited range, otherwise it's full range.

- **#define BT601_L 1**

- **#define BT709 2**

- **#define BT709_L 3**

- **#define BT2020 4**

- **#define BT2020_L 5**

- **#define IS_FULL_RANGE(fmt) ((fmt == BT601) || (fmt == BT709) || (fmt == BT2020))**

- **#define VSI_LINEAR 0**

The scaler kernel for post-process scale, for upscale, VSI_LINEAR and BOX are not supported.

- **#define LANCZOS 1**

- **#define NEAREST 2**

- **#define BI_LINEAR 3**

- **#define BICUBIC 4**

- **#define SPLINE 5**

- **#define BOX 6**

- **#define FAST_LINEAR 7**

- **#define FAST_BICUBIC 8**

- **#define TILED4x4 0**

The supported tiled mode for post-process module, which each tile size from 4x4, 4x8, 16x16, 32x8, 128x2 to 64x64 respectively.

- #define **TILED8x8** 1
- #define **TILED16x16** 2
- #define **TILED32x8** 3
- #define **TILED128x2** 4
- #define **TILED64x64** 7
- #define **DOWN_ROUND** 0

The supported center-point selected algorithm for post-process scaler.

- #define **NO_ROUND** 1
- #define **UP_ROUND** 2
- #define **YUYV** 0

The YUV packed mode for 422.

- #define **UYVY** 1
- #define **PP_YUV400** 0

The input chroma format for post-process module.

- #define **PP_YUV420** 1
- #define **PP_YUV422** 2
- #define **PP_YUV444** 3
- #define **DEC_MAX_OUT_COUNT** 6

Maximum output channels from xxxDecNexPicture(). The maximum number of output channels supported is 6.

- #define **PLANE_ALIGNMENT** 1

Extra alignment between different planes of PP buffers.

- #define **PP_IN_NV12_8BIT** 1

The input data format for standalone post-process without decoder pipeline, the input for standalone post-process is YUV or RGB.

- #define **MIN_PIC_WIDTH** 72

Minimum resolution of pictures supported by advanced decoder HW (H264/HEVC/VVC/AVS2/AVS3/VP9/AV1). Except 128x128 for AVS2, for others formats it's 72x72.

- #define **IS_RGB_4PLANE**(fmt)
- #define **IS_PIC_PACKED_RGB**(fmt)
- #define **IS_PIC_PLANAR_RGB**(fmt)
- #define **IS_PIC_RGB**(fmt)
- #define **IS_PIC_ALPHA_FORMAT**(fmt)
- #define **IS_PIC_XRGB_FORMAT**(fmt)
- #define **IS_PIC_MONOCHROME**(fmt)
- #define **IS_PIC_YCbCr420**(fmt)
- #define **IS_PIC_YCbCr422**(fmt)

- #define IS_PIC_YCbCr444(fmt)
- #define IS_PIC_PLANAR(fmt)
- #define IS_PIC_FIELD_PLANAR(fmt)
- #define IS_PIC_1010(fmt)
- #define IS_PIC_NV21(fmt)
- #define IS_PIC_SEMIPLANAR(fmt)
- #define IS_PIC_8BIT(fmt)
- #define IS_PIC_10BIT(fmt)
- #define IS_PIC_12BIT(fmt)
- #define IS_PIC_16BIT(fmt)
- #define IS_PIC_24BIT(fmt)
- #define IS_PIC_32BIT(fmt)
- #define IS_PIC_48BIT(fmt)
- #define IS_PIC_TILE(fmt)
- #define IS_PIC_RFC(fmt) ((fmt) == DEC_OUT_FRM_RFC)
- #define IS_PIC_DEC400(fmt)
- #define IS_PIC_TILED4x4(fmt)
- #define IS_PIC_REF_TILED8x8(fmt)
- #define IS_PIC_TILED8x8(fmt) ((fmt) == DEC_OUT_FRM_TILED_8X8)
- #define IS_PIC_TILED16x16(fmt) ((fmt) == DEC_OUT_FRM_TILED_16X16 || (fmt) == DEC_OUT_FRM_TILED_16X16_P010)
- #define IS_PIC_TILED16x16_COMP(fmt) ((fmt) == DEC_OUT_FRM_TILED_16X16_COMP || (fmt) == DEC_OUT_FRM_TILED_16X16_P010_COMP)
- #define IS_PIC_TILED32x8_COMP(fmt) ((fmt) == DEC_OUT_FRM_TILED_32X8_COMP || (fmt) == DEC_OUT_FRM_TILED_32X8_P010_COMP)
- #define IS_PIC_422PACKED(fmt) ((fmt) == DEC_OUT_FRM_YUV422_YUYV || (fmt) == DEC_OUT_FRM_YUV422_UYVY || (fmt) == DEC_OUT_FRM_DEC400_422PACKED)
- #define IS_PIC_TILED128x2(fmt)
- #define IS_PIC_RGB_TILED64x64(fmt)
- #define IS_PP_IN_P010(fmt) ((fmt == PP_IN_NV12_P010) || (fmt == PP_IN_TILED4x4_P010))
- #define IS_PP_IN_TILED4x4(fmt) ((fmt == PP_IN_TILED4x4) || (fmt == PP_IN_TILED4x4_P010))
- #define IS_PP_IN_RGB(fmt)
- #define IS_PP_IN_8BIT(fmt)
- #define IS_PP_IN_10BIT(fmt)
- #define IS_PP_IN_RFC(fmt) (fmt == PP_IN_RFC)
- #define UNUSED(x) (void)(x)

Macro to signal unused parameter.

- **#define** **TRANSE32**(x)

Transfer 32bit between LSB and MSB.

- **#define** **FALSE** 0

Boolean Type.

Typedef

- typedef unsigned char **u8**
- typedef signed char **i8**
- typedef unsigned short **u16**
- typedef signed short **i16**
- typedef unsigned int **u32**
- typedef signed int **i32**
- typedef long long **i64**
- typedef unsigned long long **u64**
- typedef unsigned int **u16x**

SW decoder 16 bits types.

- typedef void **DecMCStreamConsumed**(void *stream, void *p_user_data)

The prototype of the callback function for stream consumption. This callback function is called by the decoder to notify the application that data in a stream buffer has been consumed and can be reused.

*stream: the base address of the input stream buffer specified in **VCDecDecode()***

p_user_data: a application-provided pointer to user data. This is specified during decoder initialization.

4.1.1 Marco Definition

4.1.1.1 IS_RGB_4PLANE

```
#define IS_RGB_4PLANE(  
    fmt )
```

Whether the given format include 4 components, that is, A/R/G/B or X/R/G/B, with each component stored in independent plane.

4.1.1.2 IS_PIC_PACKED_RGB

```
#define IS_PIC_PACKED_RGB(  
    fmt )
```

Whether the given format is of packed RGB format, in which all components are packed together in one plane.

4.1.1.3 IS_PIC_PLANAR_RGB

```
#define IS_PIC_PLANAR_RGB(  
    fmt )
```

Value:

```
((fmt) == DEC_OUT_FRM_RGB888_P || \  
(fmt) == DEC_OUT_FRM_BGR888_P || \  
(fmt) == DEC_OUT_FRM_R16G16B16_P || \  
(fmt) == DEC_OUT_FRM_B16G16R16_P)
```

Whether the given format is of planar RGB format, with R/G/B components stored in 3 independent planes.

4.1.1.4 IS_PIC_RGB

```
#define IS_PIC_RGB(  
    fmt )
```

Whether the given format is of RGB format, either packed or planar.

4.1.1.5 IS_PIC_ALPHA_FORMAT

```
#define IS_PIC_ALPHA_FORMAT(  
    fmt )
```

Value:

```
(\  
(fmt) == DEC_OUT_FRM_ARGB888 || \  
(fmt) == DEC_OUT_FRM_ABGR888 || \  
(fmt) == DEC_OUT_FRM_A2R10G10B10 || \  
(fmt) == DEC_OUT_FRM_A2B10G10R10 || \  
(fmt) == DEC_OUT_FRM_RGBA888 || \  
(fmt) == DEC_OUT_FRM_BGRA888 || \  
(fmt) == DEC_OUT_FRM_R10G10B10A2 || \  
(fmt) == DEC_OUT_FRM_B10G10R10A2 || )
```

```
(fmt) == DEC_OUT_FRM_B10G10R10A2 || \
(fmt) == DEC_OUT_FRM_ABGR888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_ARGB888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_A2B10G10R10_TILED64X64 || \
(fmt) == DEC_OUT_FRM_A2R10G10B10_TILED64X64 || \
(fmt) == DEC_OUT_FRM_BGRA888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_RGBA888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_B10G10R10A2_TILED64X64 || \
(fmt) == DEC_OUT_FRM_R10G10B10A2_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_ABGR888 || \
(fmt) == DEC_OUT_FRM_DEC400_ARGB888 || \
(fmt) == DEC_OUT_FRM_DEC400_A2B10G10R10 || \
(fmt) == DEC_OUT_FRM_DEC400_A2R10G10B10 || \
(fmt) == DEC_OUT_FRM_DEC400_ABGR888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_ARGB888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_A2B10G10R10_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_A2R10G10B10_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_BGRA888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_RGBA888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_B10G10R10A2_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_R10G10B10A2_TILED64X64)
```

Whether the given format has alpha component.

4.1.1.6 IS_PIC_XRGB_FORMAT

```
#define IS_PIC_XRGB_FORMAT(
    fmt )
```

Value:

```
(\
(fmt) == DEC_OUT_FRM_XRGB888 || \
(fmt) == DEC_OUT_FRM_XBGR888 || \
(fmt) == DEC_OUT_FRM_XBGR888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_XRGB888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_XBGR888 || \
(fmt) == DEC_OUT_FRM_DEC400_XRGB888 || \
(fmt) == DEC_OUT_FRM_DEC400_XBGR888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_XRGB888_TILED64X64)
```

Whether the given format is of RGB format with X component.

4.1.1.7 IS_PIC_MONOCHROME

```
#define IS_PIC_MONOCHROME(
    fmt )
```

Value:

```
(\
(fmt) == DEC_OUT_FRM_MONOCHROME || \
(fmt) == DEC_OUT_FRM_YUV400TILE || \
(fmt) == DEC_OUT_FRM_YUV400TILE_PACK10 || \
(fmt) == DEC_OUT_FRM_YUV400TILE_P010 || \
```

```
(fmt) == DEC_OUT_FRM_YUV400TILE_I010 || \
(fmt) == DEC_OUT_FRM_YUV400TILE_L010 || \
(fmt) == DEC_OUT_FRM_YUV400TILE_P012 || \
(fmt) == DEC_OUT_FRM_YUV400TILE_I012 || \
(fmt) == DEC_OUT_FRM_YUV400 || \
(fmt) == DEC_OUT_FRM_YUV400_P010 || \
(fmt) == DEC_OUT_FRM_YUV400_I010 || \
(fmt) == DEC_OUT_FRM_YUV400_L010 || \
(fmt) == DEC_OUT_FRM_YUV400_P012 || \
(fmt) == DEC_OUT_FRM_YUV400_I012 || \
(fmt) == DEC_OUT_FRM_YUV400_PACK10 || \
(fmt) == DEC_OUT_FRM_DEC400_400SP || \
(fmt) == DEC_OUT_FRM_DEC400_400P || \
(fmt) == DEC_OUT_FRM_DEC400_400TILE || \
(fmt) == DEC_OUT_FRM_YUV400TILE_128X2 || \
(fmt) == DEC_OUT_FRM_YUV400TILE_128X2_P010)
```

Whether the given format is of monochroma format. That is, it has only one Y component.

4.1.1.8 IS_PIC_YCbCr420

```
#define IS_PIC_YCbCr420(
    fmt )
```

Value:

```
( \
  ((fmt) >= DEC_OUT_FRM_NV21SP && \
  (fmt) <= DEC_OUT_FRM_NV21P_1010) || \
  ((fmt) >= DEC_OUT_FRM_YUV420SP && \
  (fmt) <= DEC_OUT_FRM_YUV420P_1010))
```

Whether the given format is in YCbCr 4:2:0 sampling, that is, each of the two chroma components (U/V) has half the height and half the width of the Y component.

4.1.1.9 IS_PIC_YCbCr422

```
#define IS_PIC_YCbCr422(
    fmt )
```

Value:

```
( \
  ((fmt) >= DEC_OUT_FRM_YUV422SP && \
  (fmt) <= DEC_OUT_FRM_YUV422_UYVY) || \
  ((fmt) >= DEC_OUT_FRM_422_NV21SP && \
  (fmt) <= DEC_OUT_FRM_422_NV21P_I012))
```

Whether the given format is in YCbCr 4:2:2 sampling, that is, each of the two chroma components (U/V) has the same height and half the width of the Y component.

4.1.1.10 IS_PIC_YCbCr444

```
#define IS_PIC_YCbCr444(  
    fmt )
```

Value:

```
( \  
(fmt) >= DEC_OUT_FRM_YUV444SP && \  
(fmt) <= DEC_OUT_FRM_YUV444P_I012) || \  
(fmt) >= DEC_OUT_FRM_444_NV21SP && \  
(fmt) <= DEC_OUT_FRM_444_NV21P_I012)
```

Whether the given format is in YCbCr 4:4:4 sampling, that is, each of the two chroma components (U/V) has the same height and width of the Y component.

4.1.1.11 IS_PIC_PLANAR

```
#define IS_PIC_PLANAR(  
    fmt )
```

Whether the given format is in YUV planar format, that is, each component is stored in independent buffer.

4.1.1.12 IS_PIC_FIELD_PLANAR

```
#define IS_PIC_FIELD_PLANAR(  
    fmt )
```

Value:

```
((fmt) == DEC_OUT_FRM_PLANAR_420) || \  
(fmt) == DEC_OUT_FRM_YUV420P) || \  
(fmt) == DEC_OUT_FRM_YUV420P_P010) || \  
(fmt) == DEC_OUT_FRM_YUV420P_I010)
```

Whether the given field format is in YUV planar format, that is, each component is stored in independent buffer.

4.1.1.13 IS_PIC_1010

```
#define IS_PIC_1010(  
    fmt )
```

Value:

```
( \  
(fmt) == DEC_OUT_FRM_NV21P_1010 || \  
(fmt) == DEC_OUT_FRM_NV21SP_1010 || \  
(fmt) == DEC_OUT_FRM_YUV420P_1010 || \  
(fmt) == DEC_OUT_FRM_YUV420SP_1010 || \  
(fmt) == DEC_OUT_FRM_YUV422SP_1010 || \  
(fmt) == DEC_OUT_FRM_422_NV21SP_1010)
```

Whether the given format is in private 1010 format, that is, 3 pixels (10-bit pixel) from each component is stored in 32-bits, with 2 bits pending.

4.1.1.14 IS_PIC_NV21

```
#define IS_PIC_NV21(  
    fmt )
```

Value:

```
( \  
(fmt) == DEC_OUT_FRM_NV21TILE || \  
(fmt) == DEC_OUT_FRM_NV21TILE_PACK10 || \  
(fmt) == DEC_OUT_FRM_NV21TILE_P010 || \  
(fmt) == DEC_OUT_FRM_NV21TILE_I010 || \  
(fmt) == DEC_OUT_FRM_NV21TILE_L010 || \  
(fmt) == DEC_OUT_FRM_NV21TILE_P012 || \  
(fmt) == DEC_OUT_FRM_NV21TILE_I012 || \  
(fmt) == DEC_OUT_FRM_NV21SP || \  
(fmt) == DEC_OUT_FRM_NV21SP_PACK10 || \  
(fmt) == DEC_OUT_FRM_NV21SP_P010 || \  
(fmt) == DEC_OUT_FRM_NV21SP_I010 || \  
(fmt) == DEC_OUT_FRM_NV21SP_L010 || \  
(fmt) == DEC_OUT_FRM_NV21SP_P012 || \  
(fmt) == DEC_OUT_FRM_NV21SP_I012 || \  
(fmt) == DEC_OUT_FRM_NV21P || \  
(fmt) == DEC_OUT_FRM_NV21P_PACK10 || \  
(fmt) == DEC_OUT_FRM_NV21P_P010 || \  
(fmt) == DEC_OUT_FRM_NV21P_I010 || \  
(fmt) == DEC_OUT_FRM_NV21P_P012 || \  
(fmt) == DEC_OUT_FRM_NV21P_I012 || \  
(fmt) == DEC_OUT_FRM_NV21P_L010 )
```

Whether the given format is in NV21 format, that is, the each U/V component of a pixel is stored in interleaved mode, like V0U0V1U1V2U2...

4.1.1.15 IS_PIC_SEMIPLANAR

```
#define IS_PIC_SEMIPLANAR(  
    fmt )
```

Value:

```
(\n  
(fmt) == DEC_OUT_FRM_YUV420SP || \n  
(fmt) == DEC_OUT_FRM_YUV420SP_PACK10 || \n  
(fmt) == DEC_OUT_FRM_YUV420SP_P010 || \n  
(fmt) == DEC_OUT_FRM_YUV420SP_I010 || \n  
(fmt) == DEC_OUT_FRM_YUV420SP_L010 || \n  
(fmt) == DEC_OUT_FRM_YUV420SP_P012 || \n  
(fmt) == DEC_OUT_FRM_YUV420SP_I012 || \n  
(fmt) == DEC_OUT_FRM_NV21SP || \n  
(fmt) == DEC_OUT_FRM_NV21SP_PACK10 || \n  
(fmt) == DEC_OUT_FRM_NV21SP_P010 || \n  
(fmt) == DEC_OUT_FRM_NV21SP_I010 || \n  
(fmt) == DEC_OUT_FRM_NV21SP_L010 || \n  
(fmt) == DEC_OUT_FRM_NV21SP_P012 || \n  
(fmt) == DEC_OUT_FRM_NV21SP_I012 || \n  
(fmt) == DEC_OUT_FRM_RASTER_SCAN)
```

Whether the given format is in semi-planar format, that is, the each U/V component of a pixel is stored in interleaved mode, like V0U0V1U1V2U2... or U0V0U1V1U2V2...

4.1.1.16 IS_PIC_8BIT

```
#define IS_PIC_8BIT(  
    fmt )
```

Whether the component of each pixel is of 8 bits.

4.1.1.17 IS_PIC_10BIT

```
#define IS_PIC_10BIT(  
    fmt )
```

Value:

```
(\n  
(fmt) == DEC_OUT_FRM_YUV420SP_PACK10 || \n  
(fmt) == DEC_OUT_FRM_YUV420P_PACK10 || \n  
(fmt) == DEC_OUT_FRM_NV21SP_PACK10 || \n  
(fmt) == DEC_OUT_FRM_NV21P_PACK10 || \n  
(fmt) == DEC_OUT_FRM_YUV422SP_PACK10 || \n  
(fmt) == DEC_OUT_FRM_YUV422P_PACK10 || \n  
(fmt) == DEC_OUT_FRM_422_NV21SP_PACK10 || \n  
(fmt) == DEC_OUT_FRM_422_NV21P_PACK10 || \n  
(fmt) == DEC_OUT_FRM_YUV444SP_PACK10 || \n  
(fmt) == DEC_OUT_FRM_YUV444P_PACK10 || \n
```

```
(fmt) == DEC_OUT_FRM_444_NV21SP_PACK10 || \
(fmt) == DEC_OUT_FRM_444_NV21P_PACK10 || \
(fmt) == DEC_OUT_FRM_YUV420TILE_PACK10 || \
(fmt) == DEC_OUT_FRM_YUV400TILE_PACK10 || \
(fmt) == DEC_OUT_FRM_NV21TILE_PACK10 || \
(fmt) == DEC_OUT_FRM_YUV400_PACK10 || \
(fmt) == DEC_OUT_FRM_YUV420TILE8x8_PACK10 || \
(fmt) == DEC_OUT_FRM_YUV400TILE8x8_PACK10)
```

Whether the component of each pixel is of 10 bits.

4.1.1.18 IS_PIC_12BIT

```
#define IS_PIC_12BIT(
    fmt )
```

Value:

```
(\
(fmt) == DEC_OUT_FRM_YUV420SP_PACK12 || \
(fmt) == DEC_OUT_FRM_YUV400_PACK12 || \
(fmt) == DEC_OUT_FRM_YUV420P_PACK12 || \
(fmt) == DEC_OUT_FRM_NV21SP_PACK12 || \
(fmt) == DEC_OUT_FRM_NV21P_PACK12 || \
(fmt) == DEC_OUT_FRM_YUV422SP_PACK12 || \
(fmt) == DEC_OUT_FRM_YUV422P_PACK12 || \
(fmt) == DEC_OUT_FRM_422_NV21SP_PACK12 || \
(fmt) == DEC_OUT_FRM_422_NV21P_PACK12 || \
(fmt) == DEC_OUT_FRM_YUV444SP_PACK12 || \
(fmt) == DEC_OUT_FRM_YUV444P_PACK12 || \
(fmt) == DEC_OUT_FRM_444_NV21SP_PACK12 || \
(fmt) == DEC_OUT_FRM_444_NV21P_PACK12 || \
(fmt) == DEC_OUT_FRM_YUV420TILE8x8_PACK12)
```

Whether the component of each pixel is of 12 bits.

4.1.1.19 IS_PIC_16BIT

```
#define IS_PIC_16BIT(
    fmt )
```

Whether the component of each pixel is of 16 bits.

4.1.1.20 IS_PIC_24BIT

```
#define IS_PIC_24BIT(
    fmt )
```

Value:

```
(\
(fmt) == DEC_OUT_FRM_RGB888 || \
(fmt) == DEC_OUT_FRM_BGR888 || \
(fmt) == DEC_OUT_FRM_DEC400_BGR888 || \
(fmt) == DEC_OUT_FRM_DEC400_RGB888)
```

Whether the 3 RGB components of each pixel are store in packed 24 bits.

4.1.1.21 IS_PIC_32BIT

```
#define IS_PIC_32BIT(  
    fmt )
```

Whether the 3 RGB components with an extra alpha or X component of each pixel are store in packed 32 bits.

4.1.1.22 IS_PIC_48BIT

```
#define IS_PIC_48BIT(  
    fmt )
```

Value:

```
(\n  (fmt) == DEC_OUT_FRM_R16G16B16 || \n  (fmt) == DEC_OUT_FRM_B16G16R16)
```

Whether the 3 RGB components of each pixel are store in packed 48 bits.

4.1.1.23 IS_PIC_TILE

```
#define IS_PIC_TILE(  
    fmt )
```

Whether the component of each plane are store in tiles.

4.1.1.24 IS_PIC_RFC

```
#define IS_PIC_RFC(  
    fmt) ((fmt) == DEC_OUT_FRM_RFC)
```

Whether the component of each plane are store in compressed format from RFC (Reference Frame Compression).

4.1.1.25 IS_PIC_DEC400

```
#define IS_PIC_DEC400(  
    fmt )
```

Value:

```
(\n  
(fmt) == DEC_OUT_FRM_DEC400_400SP || \n  
(fmt) == DEC_OUT_FRM_DEC400_420SP || \n  
(fmt) == DEC_OUT_FRM_DEC400_400P || \n  
(fmt) == DEC_OUT_FRM_DEC400_420P || \n  
(fmt) == DEC_OUT_FRM_DEC400_400TILE || \n  
(fmt) == DEC_OUT_FRM_DEC400_420TILE || \n  
(fmt) == DEC_OUT_FRM_DEC400_BGR888 || \n  
(fmt) == DEC_OUT_FRM_DEC400_RGB888 || \n  
(fmt) == DEC_OUT_FRM_DEC400_ABGR888 || \n  
(fmt) == DEC_OUT_FRM_DEC400_ARGB888 || \n  
(fmt) == DEC_OUT_FRM_DEC400_A2B10G10R10 || \n  
(fmt) == DEC_OUT_FRM_DEC400_A2R10G10B10 || \n  
(fmt) == DEC_OUT_FRM_DEC400_XBGR888 || \n  
(fmt) == DEC_OUT_FRM_DEC400_XRGB888 || \n  
(fmt) == DEC_OUT_FRM_DEC400_ABGR888_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_ARGB888_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_A2B10G10R10_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_A2R10G10B10_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_XBGR888_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_XRGB888_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_BGRA888_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_RGBA888_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_B10G10R10A2_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_R10G10B10A2_TILED64X64 || \n  
(fmt) == DEC_OUT_FRM_DEC400_422PACKED)
```

Whether the component of each plane are store in compressed format from DEC400.

4.1.1.26 IS_PIC_TILED4x4

```
#define IS_PIC_TILED4x4(  
    fmt )
```

Value:

```
(\n  
(fmt) == DEC_OUT_FRM_TILED_4X4 || \n  
(fmt) == DEC_OUT_FRM_YUV420TILE || \n  
(fmt) == DEC_OUT_FRM_YUV420TILE_PACK10 || \n  
(fmt) == DEC_OUT_FRM_YUV420TILE_P010 || \n  
(fmt) == DEC_OUT_FRM_YUV420TILE_I010 || \n  
(fmt) == DEC_OUT_FRM_YUV420TILE_L010 || \n  
(fmt) == DEC_OUT_FRM_YUV420TILE_P012 || \n  
(fmt) == DEC_OUT_FRM_YUV420TILE_I012 || \n  
(fmt) == DEC_OUT_FRM_YUV400TILE || \n  
(fmt) == DEC_OUT_FRM_YUV400TILE_PACK10 || \n  
(fmt) == DEC_OUT_FRM_YUV400TILE_P010 || \n  
(fmt) == DEC_OUT_FRM_YUV400TILE_I010 || \n  
(fmt) == DEC_OUT_FRM_YUV400TILE_L010 || \n  
(fmt) == DEC_OUT_FRM_YUV400TILE_P012 || \n  
(fmt) == DEC_OUT_FRM_YUV400TILE_I012 || \n
```

```
(fmt) == DEC_OUT_FRM_NV21TILE || \
(fmt) == DEC_OUT_FRM_NV21TILE_PACK10 || \
(fmt) == DEC_OUT_FRM_NV21TILE_P012 || \
(fmt) == DEC_OUT_FRM_NV21TILE_I010 || \
(fmt) == DEC_OUT_FRM_NV21TILE_I012 || \
(fmt) == DEC_OUT_FRM_NV21TILE_P010)
```

Whether the component of each plane are store in 4x4 tiles.

4.1.1.27 IS_PIC_REF_TILED8x8

```
#define IS_PIC_REF_TILED8x8(
    fmt )
```

Value:

```
(\
(fmt) == DEC_OUT_FRM_YUV420TILE8x8 || \
(fmt) == DEC_OUT_FRM_YUV420TILE8x8_PACK10 || \
(fmt) == DEC_OUT_FRM_YUV420TILE8x8_PACK12 || \
(fmt) == DEC_OUT_FRM_YUV400TILE8x8 || \
(fmt) == DEC_OUT_FRM_YUV400TILE8x8_PACK10)
```

Whether the component of each plane of reference buffer are store in 8x8 tiles.

4.1.1.28 IS_PIC_TILED8x8

```
#define IS_PIC_TILED8x8(
    fmt ) ((fmt) == DEC_OUT_FRM_TILED_8X8)
```

Whether the component of each plane are store in 8x8 tiles.

4.1.1.29 IS_PIC_TILED16x16

```
#define IS_PIC_TILED16x16(
    fmt ) ((fmt) == DEC_OUT_FRM_TILED_16X16 || (fmt) == DEC_OUT_FRM_TILED_16X16_P010)
```

Whether the component of each plane are store in 16x16 tiles.

4.1.1.30 IS_PIC_TILED16x16_COMP

```
#define IS_PIC_TILED16x16_COMP(
    fmt ) ((fmt) == DEC_OUT_FRM_TILED_16X16_COMP || (fmt) == DEC_OUT_FRM_TILED_16X16_P010_COMP)
```

Whether the component of each plane are store in compressed 16x16 tiles (always from third-party compressor)

4.1.1.31 IS_PIC_TILED32x8_COMP

```
#define IS_PIC_TILED32x8_COMP(  
    fmt ) ((fmt) == DEC_OUT_FRM_TILED_32X8_COMP || (fmt) == DEC_OUT_FRM_TILED_32X8_P010_COMP)
```

Whether the component of each plane are store in compressed 32x8 tiles (always from third-party compressor)

4.1.1.32 IS_PIC_422PACKED

```
#define IS_PIC_422PACKED(  
    fmt ) ((fmt) == DEC_OUT_FRM_YUV422_YUYV || (fmt) == DEC_OUT_FRM_YUV422_UYVY ||  
    (fmt) == DEC_OUT_FRM_DEC400_422PACKED)
```

Whether the 3 components are in YUV422 sampling and stored in packed mode, that is, in YUYV or UYVY, or corresponding DEC400 output.

4.1.1.33 IS_PIC_TILED128x2

```
#define IS_PIC_TILED128x2(  
    fmt )
```

Value:

```
((fmt) == DEC_OUT_FRM_YUV400TILE_128X2 || \  
(fmt) == DEC_OUT_FRM_YUV420TILE_128X2 || \  
(fmt) == DEC_OUT_FRM_YUV400TILE_128X2_P010 || \  
(fmt) == DEC_OUT_FRM_YUV420TILE_128X2_P010)
```

Whether the components of each plane are store in 128x2 tiles.

4.1.1.34 IS_PIC_RGB_TILED64x64

```
#define IS_PIC_RGB_TILED64x64(  
    fmt )
```

Value:

```
( \  
(fmt) == DEC_OUT_FRM_ABGR888_TILED64X64 || \  
(fmt) == DEC_OUT_FRM_ARGB888_TILED64X64 || \  
(fmt) == DEC_OUT_FRM_A2B10G10R10_TILED64X64 || \  
(fmt) == DEC_OUT_FRM_A2R10G10B10_TILED64X64 || \  
(fmt) == DEC_OUT_FRM_XBGR888_TILED64X64 || \  
(fmt) == DEC_OUT_FRM_XRGB888_TILED64X64 || )
```

```
(fmt) == DEC_OUT_FRM_XRGB888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_BGRA888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_RGBA888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_B10G10R10A2_TILED64X64 || \
(fmt) == DEC_OUT_FRM_R10G10B10A2_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_ABGR888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_ARGB888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_A2B10G10R10_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_A2R10G10B10_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_XBGR888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_XRGB888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_BGRA888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_RGBA888_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_B10G10R10A2_TILED64X64 || \
(fmt) == DEC_OUT_FRM_DEC400_R10G10B10A2_TILED64X64)
```

Whether the all 3 RGB components are packed and stored in 64x64 tiles.

4.1.1.35 IS_PP_IN_P010

```
#define IS_PP_IN_P010(
    fmt ) ((fmt == PP_IN_NV12_P010) || (fmt == PP_IN_TILED4x4_P010))
```

Whether each component of standalone input buffer are in P010 16-bit format.

4.1.1.36 IS_PP_IN_TILED4x4

```
#define IS_PP_IN_TILED4x4(
    fmt ) ((fmt == PP_IN_TILED4x4) || (fmt == PP_IN_TILED4x4_P010))
```

Whether each component of standalone input buffer are in 4x4 tiles.

4.1.1.37 IS_PP_IN_RGB

```
#define IS_PP_IN_RGB(
    fmt )
```

Value:

```
((fmt == PP_IN_RGB888) || \
 (fmt == PP_IN_BGR888) || \
 (fmt == PP_IN_ARGB888) || \
 (fmt == PP_IN_ABGR888) || \
 (fmt == PP_IN_A2R10G10B10) || \
 (fmt == PP_IN_A2B10G10R10) || \
 (fmt == PP_IN_X2R10G10B10) || \
 (fmt == PP_IN_X2B10G10R10))
```

Whether standalone input buffer are in 4x4 RGB packed format.

4.1.1.38 IS_PP_IN_8BIT

```
#define IS_PP_IN_8BIT(  
    fmt )
```

Value:

```
((fmt == PP_IN_NV12_8BIT) || \  
(fmt == PP_IN_TILED4x4) || \  
(fmt == PP_IN_RGB888) || \  
(fmt == PP_IN_BGR888) || \  
(fmt == PP_IN_ARGB888) || \  
(fmt == PP_IN_ABGR888))
```

Whether each component of standalone input buffer are in 8-bit format.

4.1.1.39 IS_PP_IN_10BIT

```
#define IS_PP_IN_10BIT(  
    fmt )
```

Value:

```
((fmt == PP_IN_NV12_P010) || \  
(fmt == PP_IN_TILED4x4_P010) || \  
(fmt == PP_IN_A2R10G10B10) || \  
(fmt == PP_IN_A2B10G10R10) || \  
(fmt == PP_IN_X2R10G10B10) || \  
(fmt == PP_IN_X2B10G10R10))
```

Whether each component of standalone input buffer is of 10 valid bits.

4.1.1.40 IS_PP_IN_RFC

```
#define IS_PP_IN_RFC(  
    fmt ) (fmt == PP_IN_RFC)
```

Whether each component of standalone input buffer is in RFC format.

4.1.1.41 TRANSE32

```
#define TRANSE32(  
    x )
```

Value:

```
((((u32)(x) & 0xff000000) >> 24) | \  
((u32)(x) & 0x00ff0000) >> 8) | \  
((u32)(x) & 0x0000ff00) << 8) | \  
((u32)(x) & 0x000000ff) << 24))
```


4.1.2 Enumeration Type

4.1.2.1 DecDpbMode

enum `DecDpbMode`

Defines content storage modes in DPB.

Enumerator

DEC_DPB_FRAME	(Default) Data in DPB is stored by frame.
DEC_DPB_INTERLACED_FIELD	Data in DPB is stored by interlaced field.

4.1.2.2 DecDecoderMode

enum `DecDecoderMode`

Defines decoder working modes.

Enumerator

DEC_NORMAL	The normal decoding mode, in which the hardware is not enabled until the complete data of a frame is ready. The minimum hardware decoding unit is a frame.
DEC_LOW_LATENCY	The low-latency decoding mode, in which the hardware can be enabled and then wait for input stream data. The minimum hardware decoding unit is 256 bytes.
DEC_SECURITY	(Obsoleted) The security playback mode.
DEC_PARTIAL_DECODING	A special decoding mode: don't write recon data, don't write reference data, don't write MV, don't write pp out for non-reference picture. [Not supported].

Enumerator

DEC_INTRA_ONLY	The intra-only decoding mode, which decodes intra frames only.
DEC_HEIF	The HEIF mode, in which the decoder is fed with HEIF files as its input.

4.1.2.3 DecAuxInfo

enum `DecAuxInfo`

Defines flags to indicate whether auxiliary information is present.

Enumerator

DEC_AUX_NONE	No auxiliary information is present.
DEC_AUX_QP	QP information is present.
DEC_AUX_MV	MV information is present.

4.1.2.4 DelogoMode

enum `DelogoMode`

Defines delogo filter modes.

Enumerator

PIXEL_NO_DELOGO	Disables the delogo filter.
PIXEL_REPLACE	Enables the delogo filter in replacement mode.
PIXEL_INTERPOLATION	Enables the delogo filter in interpolation mode.

4.1.2.5 DecPictureFormat

enum DecPictureFormat

Defines output picture formats.

For details about color formats, see *Hantro VC9000D Series Memory Buffer and Format Organization*.

Enumerator

DEC_OUT_FRM_TILED_4X4	Tiled 4x4 format.
DEC_OUT_FRM_TILED_8X4	(Obsoleted).
DEC_OUT_FRM_RASTER_SCAN	YUV420SP8b_Raster_AX.
DEC_OUT_FRM_PLANAR_420	YUV420P8b_Raster_AX.
DEC_OUT_FRM_TILED_8X8	Tiled 8x8 format.
DEC_OUT_FRM_TILED_16X16	Tiled 16x16 format.
DEC_OUT_FRM_TILED_16X16_P010	Tiled 16x16 P010 format, 10&12 bit.
DEC_OUT_FRM_MONOCHROME	Y8b_Raster_AX.
DEC_OUT_FRM_RFC	Reference frame compression (RFC) compressed format.
DEC_OUT_FRM_TILED_32X8_COMP	Third-party compressed 16x16 tiled format.
DEC_OUT_FRM_TILED_32X8_P010_COMP	Third-party compressed 16x16 tiled P010 format, 10&12 bit.
DEC_OUT_FRM_TILED_16X16_COMP	Third-party compressed 16x16 tiled format.
DEC_OUT_FRM_TILED_16X16_P010_COMP	Third-party compressed 16x16 tiled P010 format, 10&12 bit.
DEC_OUT_FRM_DEC400_400SP	DEC400 compressed YUV400 semi-planar.
DEC_OUT_FRM_DEC400_420SP	DEC400 compressed YUV420 semi-planar.
DEC_OUT_FRM_DEC400_400TILE	DEC400 compressed YUV400 tile4x4.
DEC_OUT_FRM_DEC400_420TILE	DEC400 compressed YUV420 tile4x4.
DEC_OUT_FRM_DEC400_400P	DEC400 compressed YUV400 planar.
DEC_OUT_FRM_DEC400_420P	DEC400 compressed YUV420 planar.
DEC_OUT_FRM_DEC400_422PACKED	DEC400 output YUV422 8bit packed.
DEC_OUT_FRM_DEC400_BGR888	DEC400 output BGR888.
DEC_OUT_FRM_DEC400_RGB888	DEC400 output RGB888.
DEC_OUT_FRM_DEC400_ABGR888	DEC400 output ABGR888.
DEC_OUT_FRM_DEC400_ARGB888	DEC400 output ARGB888.
DEC_OUT_FRM_DEC400_A2B10G10R10	DEC400 output A2B10G10R10.

Enumerator

DEC_OUT_FRM_DEC400_A2R10G10B10	DEC400 output A2R10G10B10.
DEC_OUT_FRM_DEC400_XBGR888	DEC400 output XBGR888.
DEC_OUT_FRM_DEC400_XRGB888	DEC400 output XRGB888.
DEC_OUT_FRM_DEC400_ABGR888_TILED64X64	DEC400 output ABGR888_TILED64X64.
DEC_OUT_FRM_DEC400_ARGB888_TILED64X64	DEC400 output ARGB888_TILED64X64.
DEC_OUT_FRM_DEC400_A2B10G10R10_TILE64X64	DEC400 output A2B10G10R10_TILE64X64.
DEC_OUT_FRM_DEC400_A2R10G10B10_TILE64X64	DEC400 output A2R10G10B10_TILE64X64.
DEC_OUT_FRM_DEC400_XBGR888_TILED64X64	DEC400 output XBGR888_TILED64X64.
DEC_OUT_FRM_DEC400_XRGB888_TILED64X64	DEC400 output XRGB888_TILED64X64.
DEC_OUT_FRM_DEC400_BGRA888_TILED64X64	DEC400 output BGRA888_TILED64X64.
DEC_OUT_FRM_DEC400_RGBA888_TILED64X64	DEC400 output RGBA888_TILED64X64.
DEC_OUT_FRM_DEC400_B10G10R10A2_TILE64X64	DEC400 output B10G10R10A2_TILE64X64.
DEC_OUT_FRM_DEC400_R10G10B10A2_TILE64X64	DEC400 output R10G10B10A2_TILE64X64.
DEC_OUT_FRM_YUV420TILE	Uncompressed YUV420SP8b_YuvSp4x4_AX for output to DPB.
DEC_OUT_FRM_YUV420TILE_PACK10	Uncompressed YUV420SP10b_YuvSp4x4_AX for output to DPB.
DEC_OUT_FRM_YUV420TILE_PACK12	Uncompressed YUV420SP12b_YuvSp4x4_AX for output to DPB.
DEC_OUT_FRM_YUV420TILE8x8	Uncompressed YUV420SP8b_YuvSp8x8_AX for output to DPB.

Enumerator

DEC_OUT_FRM_YUV420TILE8x8_PACK10	Uncompressed YUV420SP10b_YuvSp8x8_AX for output to DPB.
DEC_OUT_FRM_YUV420TILE8x8_PACK12	Uncompressed YUV420SP12b_YuvSp8x8_AX for output to DPB.
DEC_OUT_FRM_YUV420TILE_P010	YUV420SP10bWH_YuvSp4x4_AX.
DEC_OUT_FRM_YUV420TILE_I010	YUV420SP10bWL_YuvSp4x4_AX I010.
DEC_OUT_FRM_YUV420TILE_L010	YUV420SP10bWL_YuvSp4x4_AX L010.
DEC_OUT_FRM_YUV420TILE_P012	YUV420SP12bWH_YuvSp4x4_AX.
DEC_OUT_FRM_YUV420TILE_I012	YUV420SP12bWL_YuvSp4x4_AX I012.
DEC_OUT_FRM_YUV420TILE_128X2	YUV420SP8b_YuvSp128x2_AX.
DEC_OUT_FRM_YUV420TILE_128X2_P010	YUV420SP10bWH_YuvSp128x2_AX.
DEC_OUT_FRM_YUV420SP	YUV420SP8b_Raster_AX.
DEC_OUT_FRM_YUV420SP_PACK10	YUV420SP10b_Raster_AX.
DEC_OUT_FRM_YUV420SP_PACK12	(Obsoleted) YUV420SP12b_Raster_AX.
DEC_OUT_FRM_YUV420SP_P010	YUV420SP10bWH_Raster_AX.
DEC_OUT_FRM_YUV420SP_I010	YUV420SP10bWL_Raster_AX I010.
DEC_OUT_FRM_YUV420SP_L010	YUV420SP10bWL_Raster_AX L010.
DEC_OUT_FRM_YUV420SP_P012	YUV420SP12bWH_Raster_AX.
DEC_OUT_FRM_YUV420SP_I012	YUV420SP12bWL_Raster_AX I012.
DEC_OUT_FRM_YUV420SP_1010	YUV420SP10bDWL_Raster_AX.
DEC_OUT_FRM_YUV420P	YUV420P8b_Raster_AX.
DEC_OUT_FRM_YUV420P_PACK10	YUV420P10b_Raster_AX.
DEC_OUT_FRM_YUV420P_PACK12	YUV420P12b_Raster_AX.
DEC_OUT_FRM_YUV420P_P010	YUV420P10bWH_Raster_AX.
DEC_OUT_FRM_YUV420P_I010	YUV420P10bWL_Raster_AX I010.
DEC_OUT_FRM_YUV420P_L010	YUV420P10bWL_Raster_AX L010.
DEC_OUT_FRM_YUV420P_P012	YUV420P12bWH_Raster_AX.
DEC_OUT_FRM_YUV420P_I012	YUV420P12bWL_Raster_AX I012.
DEC_OUT_FRM_YUV420P_1010	YUV420P10bDWL_Raster_AX.
DEC_OUT_FRM_YUV400TILE	Uncompressed Y8b_Tile4x4_AX for output to DPB.

Enumerator

DEC_OUT_FRM_YUV400TILE_PACK10	Uncompressed Y10b_Tile4x4_AX for output to DPB. Uncompressed Y8b_Tile8x8_AX for output to DPB.
DEC_OUT_FRM_YUV400TILE8x8	Uncompressed Y10b_Tile8x8_AX for output to DPB.
DEC_OUT_FRM_YUV400TILE_P010	Y10bWH_Tile4x4_AX.
DEC_OUT_FRM_YUV400TILE_I010	Y10bWL_Tile4x4_AX I010.
DEC_OUT_FRM_YUV400TILE_L010	Y10bWL_Tile4x4_AX L010.
DEC_OUT_FRM_YUV400TILE_P012	(Obsoleted) Y12bWH_Tile4x4_AX.
DEC_OUT_FRM_YUV400TILE_I012	(Obsoleted) Y12bWL_Tile4x4_AX I012.
DEC_OUT_FRM_YUV400	Y8b_Raster_AX.
DEC_OUT_FRM_YUV400_PACK10	Y10b_Raster_AX.
DEC_OUT_FRM_YUV400_PACK12	(Obsoleted) Y12b_Raster_AX.
DEC_OUT_FRM_YUV400_P010	Y10bWH_Raster_AX.
DEC_OUT_FRM_YUV400_I010	Y10bWL_Raster_AX I010.
DEC_OUT_FRM_YUV400_L010	Y10bWL_Raster_AX L010.
DEC_OUT_FRM_YUV400_P012	Y12bWH_Raster_AX.
DEC_OUT_FRM_YUV400_I012	Y12bWL_Raster_AX I012.
DEC_OUT_FRM_YUV400TILE_128X2	Y8b_Tile128x2_AX. Y10bWH_Tile128x2_AX.
DEC_OUT_FRM_YUV400TILE_128X2_P010	
DEC_OUT_FRM_YUV422SP	YUV422SP8b_Raster_AX.
DEC_OUT_FRM_YUV422SP_PACK10	YUV422SP10b_Raster_AX.
DEC_OUT_FRM_YUV422SP_PACK12	(Obsoleted) YUV422SP12b_Raster_AX.
DEC_OUT_FRM_YUV422SP_P010	YUV422SP10bWH_Raster_AX.
DEC_OUT_FRM_YUV422SP_I010	YUV422SP10bWL_Raster_AX I010.
DEC_OUT_FRM_YUV422SP_L010	YUV422SP10bWL_Raster_AX L010.
DEC_OUT_FRM_YUV422SP_P012	(Obsoleted) YUV422SP12bWH_Raster_AX.
DEC_OUT_FRM_YUV422SP_I012	(Obsoleted) YUV422SP12bWL_Raster_AX I012.
DEC_OUT_FRM_YUV422SP_1010	YUV422SP10bDWL_Raster_AX.
DEC_OUT_FRM_YUV422P	(Obsoleted) YUV422P8b_Raster_AX.
DEC_OUT_FRM_YUV422P_PACK10	(Obsoleted) YUV422P10b_Raster_AX.
DEC_OUT_FRM_YUV422P_PACK12	(Obsoleted) YUV422P12b_Raster_AX.
DEC_OUT_FRM_YUV422P_P010	YUV422P10bWH_Raster_AX.

Enumerator

DEC_OUT_FRM_YUV422P_I010	(Obsoleted) YUV422P10bWL_Raster_AX I010.
DEC_OUT_FRM_YUV422P_L010	(Obsoleted) YUV422P10bWL_Raster_AX L010.
DEC_OUT_FRM_YUV422P_P012	(Obsoleted) YUV422P12bWH_Raster_AX.
DEC_OUT_FRM_YUV422P_I012	(Obsoleted) YUV422P12bWL_Raster_AX I012.
DEC_OUT_FRM_YUV422_YUYV	YUV422YUYV8b_Raster_AX.
DEC_OUT_FRM_YUV422_UYVY	YUV422UYVY8b_Raster_AX.
DEC_OUT_FRM_YUV440	(Obsoleted) YUV440 8-bit raster-scan.
DEC_OUT_FRM_YUV411SP	(Obsoleted) YUV411SP8b_Raster_AX.
DEC_OUT_FRM_YUV444SP	(Obsoleted) YUV444SP8b_Raster_AX.
DEC_OUT_FRM_YUV444SP_PACK10	(Obsoleted) YUV444SP10b_Raster_AX.
DEC_OUT_FRM_YUV444SP_PACK12	(Obsoleted) YUV444SP12b_Raster_AX.
DEC_OUT_FRM_YUV444SP_P010	(Obsoleted) YUV444SP10bWH_Raster_AX.
DEC_OUT_FRM_YUV444SP_I010	(Obsoleted) YUV444SP10bWL_Raster_AX I010.
DEC_OUT_FRM_YUV444SP_L010	(Obsoleted) YUV444SP10bWL_Raster_AX L010.
DEC_OUT_FRM_YUV444SP_P012	(Obsoleted) YUV444SP12bWH_Raster_AX.
DEC_OUT_FRM_YUV444SP_I012	(Obsoleted) YUV444SP12bWL_Raster_AX I012.
DEC_OUT_FRM_YUV444SP_I010	(Obsoleted) YUV444SP10bDWL_Raster_AX.
DEC_OUT_FRM_YUV444P	YUV444P8b_Raster_AX.
DEC_OUT_FRM_YUV444P_PACK10	(Obsoleted) YUV444P10b_Raster_AX.
DEC_OUT_FRM_YUV444P_PACK12	(Obsoleted) YUV444P12b_Raster_AX.
DEC_OUT_FRM_YUV444P_P010	YUV444P10bWH_Raster_AX.
DEC_OUT_FRM_YUV444P_I010	(Obsoleted) YUV444P10bWL_Raster_AX I010.
DEC_OUT_FRM_YUV444P_L010	(Obsoleted) YUV444P10bWL_Raster_AX L010.
DEC_OUT_FRM_YUV444P_P012	(Obsoleted) YUV444P12bWH_Raster_AX.
DEC_OUT_FRM_YUV444P_I012	(Obsoleted) YUV444P12bWL_Raster_AX I012.
DEC_OUT_FRM_NV21TILE	YVU420SP8b_YuvSp4x4_AX.
DEC_OUT_FRM_NV21TILE_PACK10	YVU420SP10b_YuvSp4x4_AX.

Enumerator

DEC_OUT_FRM_NV21TILE_P010	YVU420SP10bWH_YuvSp4x4_AX.
DEC_OUT_FRM_NV21TILE_I010	YVU420SP10bWL_YuvSp4x4_AX I010.
DEC_OUT_FRM_NV21TILE_L010	YVU420SP10bWL_YuvSp4x4_AX L010.
DEC_OUT_FRM_NV21TILE_P012	YVU420SP12bWH_YuvSp4x4_AX.
DEC_OUT_FRM_NV21TILE_I012	YVU420SP12bWL_YuvSp4x4_AX I012.
DEC_OUT_FRM_NV21SP	YVU420SP8b_Raster_AX.
DEC_OUT_FRM_NV21SP_PACK10	YVU420SP10b_Raster_AX.
DEC_OUT_FRM_NV21SP_PACK12	YVU420SP12b_Raster_AX.
DEC_OUT_FRM_NV21SP_P010	YVU420SP10bWH_Raster_AX.
DEC_OUT_FRM_NV21SP_I010	YVU420SP10bWL_Raster_AX I010.
DEC_OUT_FRM_NV21SP_L010	YVU420SP10bWL_Raster_AX L010.
DEC_OUT_FRM_NV21SP_P012	(Obsoleted) YVU420SP12bWH_Raster_AX.
DEC_OUT_FRM_NV21SP_I012	(Obsoleted) YVU420SP12bWL_Raster_AX I012.
DEC_OUT_FRM_NV21SP_1010	YVU420SP10bDWL_Raster_AX.
DEC_OUT_FRM_NV21P	YVU420P8b_Raster_AX.
DEC_OUT_FRM_NV21P_PACK10	YVU420P10b_Raster_AX.
DEC_OUT_FRM_NV21P_PACK12	(Obsoleted) YVU420P12b_Raster_AX.
DEC_OUT_FRM_NV21P_P010	YVU420P10bWH_Raster_AX.
DEC_OUT_FRM_NV21P_I010	YVU420P10bWL_Raster_AX I010.
DEC_OUT_FRM_NV21P_L010	YVU420P10bWL_Raster_AX L010.
DEC_OUT_FRM_NV21P_P012	YVU420SP12bWH_Raster_AX.
DEC_OUT_FRM_NV21P_I012	YVU420P12bWL_Raster_AX I012.
DEC_OUT_FRM_NV21P_1010	YVU420P10bDWL_Raster_AX.
DEC_OUT_FRM_422_NV21SP	YVU422SP8b_Raster_AX.
DEC_OUT_FRM_422_NV21SP_PACK10	YVU422SP10b_Raster_AX.
DEC_OUT_FRM_422_NV21SP_PACK12	YVU422SP12b_Raster_AX.
DEC_OUT_FRM_422_NV21SP_P010	YVU422SP10bWH_Raster_AX.
DEC_OUT_FRM_422_NV21SP_I010	YVU422SP10bWL_Raster_AX I010.
DEC_OUT_FRM_422_NV21SP_L010	YVU422SP10bWL_Raster_AX L010.
DEC_OUT_FRM_422_NV21SP_P012	(Obsoleted) YVU422SP12bWH_Raster_AX.
DEC_OUT_FRM_422_NV21SP_I012	(Obsoleted) YVU422SP12bWL_Raster_AX I012.
DEC_OUT_FRM_422_NV21SP_1010	YVU422SP10bDWL_Raster_AX.
DEC_OUT_FRM_422_NV21P	YVU422P8b_Raster_AX.
DEC_OUT_FRM_422_NV21P_PACK10	YVU422P10b_Raster_AX.

Enumerator

DEC_OUT_FRM_422_NV21P_PACK12	(Obsoleted) YVU422P12b_Raster_AX.
DEC_OUT_FRM_422_NV21P_P010	YVU422P10bWH_Raster_AX.
DEC_OUT_FRM_422_NV21P_I010	(Obsoleted) YVU422P10bWL_Raster_AX I010.
DEC_OUT_FRM_422_NV21P_L010	(Obsoleted) YVU422P10bWL_Raster_AX L010.
DEC_OUT_FRM_422_NV21P_P012	(Obsoleted) YVU422P12bWH_Raster_AX.
DEC_OUT_FRM_422_NV21P_I012	(Obsoleted) YVU422P12bWL_Raster_AX I012.
DEC_OUT_FRM_444_NV21SP	YVU444SP8b_Raster_AX.
DEC_OUT_FRM_444_NV21SP_PACK10	YVU444SP10b_Raster_AX.
DEC_OUT_FRM_444_NV21SP_PACK12	(Obsoleted) YVU444SP12b_Raster_AX.
DEC_OUT_FRM_444_NV21SP_P010	YVU444SP10bWH_Raster_AX.
DEC_OUT_FRM_444_NV21SP_I010	(Obsoleted) YVU422SP10bWL_Raster_AX I010.
DEC_OUT_FRM_444_NV21SP_L010	(Obsoleted) YVU422SP10bWL_Raster_AX L010.
DEC_OUT_FRM_444_NV21SP_P012	(Obsoleted) YVU444SP12bWH_Raster_AX.
DEC_OUT_FRM_444_NV21SP_I012	(Obsoleted) YVU444SP12bWL_Raster_AX I012.
DEC_OUT_FRM_444_NV21P	YVU444P8b_Raster_AX.
DEC_OUT_FRM_444_NV21P_PACK10	YVU444P10b_Raster_AX.
DEC_OUT_FRM_444_NV21P_PACK12	YVU444P12b_Raster_AX.
DEC_OUT_FRM_444_NV21P_P010	YVU444P10bWH_Raster_AX.
DEC_OUT_FRM_444_NV21P_I010	YVU444P10bWL_Raster_AX I010.
DEC_OUT_FRM_444_NV21P_L010	YVU444P10bWL_Raster_AX L010.
DEC_OUT_FRM_444_NV21P_P012	YVU444P12bWH_Raster_AX.
DEC_OUT_FRM_444_NV21P_I012	YVU444P12bWL_Raster_AX I012.
DEC_OUT_FRM_RGB888_P	RGB888P_Raster_AX.
DEC_OUT_FRM_BGR888	BGR888_Raster_AX.
DEC_OUT_FRM_BGR888_P	BGR888P_Raster_AX.
DEC_OUT_FRM_RGB888	RGB888_Raster_AX.
DEC_OUT_FRM_R16G16B16_P	RGB16I16I16IP_Raster_AX.
DEC_OUT_FRM_B16G16R16	BGR16I16I16I_Raster_AX.
DEC_OUT_FRM_B16G16R16_P	BGR16I16I16IP_Raster_AX.
DEC_OUT_FRM_R16G16B16	RGB16I16I16_Raster_AX.

Enumerator

DEC_OUT_FRM_ABGR888	ABGR8888_Raster_AX.
DEC_OUT_FRM_ARGB888	ARGB8888_Raster_AX.
DEC_OUT_FRM_A2B10G10R10	ABGR2101010_Raster_AX.
DEC_OUT_FRM_A2R10G10B10	ARGB2101010_Raster_AX.
DEC_OUT_FRM_XBGR888	XBGR8888_Raster_AX.
DEC_OUT_FRM_XRGB888	XRGB8888_Raster_AX.
DEC_OUT_FRM_BGRA888	BGRA8888_Raster_AX.
DEC_OUT_FRM_RGBA888	RGBA8888_Raster_AX.
DEC_OUT_FRM_B10G10R10A2	BGRA1010102_Raster_AX.
DEC_OUT_FRM_R10G10B10A2	RGBA1010102_Raster_AX.
DEC_OUT_FRM_ABGR888_TILED64X64	ABGR8888_Tile64x64_AX.
DEC_OUT_FRM_ARGB888_TILED64X64	ARGB8888_Tile64x64_AX.
	ABGR2101010_Tile64x64_AX.
DEC_OUT_FRM_A2B10G10R10_TILED64X64	
	ARGB2101010_Tile64x64_AX.
DEC_OUT_FRM_A2R10G10B10_TILED64X64	
DEC_OUT_FRM_XBGR888_TILED64X64	XBGR8888_Tile64x64_AX.
DEC_OUT_FRM_XRGB888_TILED64X64	XRGB8888_Tile64x64_AX.
DEC_OUT_FRM_BGRA888_TILED64X64	BGRA stored in Tiled 64x64 format.
DEC_OUT_FRM_RGBA888_TILED64X64	RGBA stored in Tiled 64x64 format.
	B10G10R10A2 stored in Tiled 64x64 format.
DEC_OUT_FRM_B10G10R10A2_TILED64X64	
	R10G10B10A2 stored in Tiled 64x64 format.
DEC_OUT_FRM_R10G10B10A2_TILED64X64	

4.1.2.6 DecPicCodingType

```
enum DecPicCodingType
```

Defines picture coding types.

Enumerator

DEC_PIC_TYPE_I	Intra coding picture.
----------------	-----------------------

Enumerator

DEC_PIC_TYPE_P	P picture: a picture using intra prediction or using inter prediction with at most one motion vector and reference index to predict the sample values of each block.
DEC_PIC_TYPE_B	B picture: a picture using intra prediction or using inter prediction with at most two motion vectors and reference indeices to predict the sample values of each block.
DEC_PIC_TYPE_D	D-Picture in MPEG2.
DEC_PIC_TYPE_FI	FORCED_INTRAPIC defined in RealVideo.
DEC_PIC_TYPE_BI	BI picture: A B picture where all the macroblocks are intra-coded, which is defined in VC-1.

4.1.2.7 DecErrorHandling

enum `DecErrorHandling`

Defines error handling modes.

Enumerator

DEC_EC_FRAME_NO_ERROR	Clarity first mode, which outputs only good pictures. Here, a good picture means no stream error is detected when decoding, and all the referred pictures are good pictures too.
DEC_EC_FRAME_IGNORE_ERROR	Fluency first mode, which decodes as many pictures as possible and outputs all pictures (include erroneous ones).
DEC_EC_FRAME_TOLERANT_ERROR	Balanced mode: tradeoff between fluency and clarity. That is, some erronous pictures will be treated as "good" ones, which is determined by error_ratio. The default error_ratio is 10%. Those "good" pictures will be used as reference and output as usual. Beside, a bad reference picture will be replaced by a nearest good picture when decoding.

4.1.2.8 DecPicAlignment

enum `DecPicAlignment`

Defines stride alignments.

Enumerator

DEC_ALIGN_1B	Alignment: 1 byte.
DEC_ALIGN_8B	8-byte alignment.
DEC_ALIGN_16B	16-byte alignment.
DEC_ALIGN_32B	32-byte alignment.
DEC_ALIGN_64B	64-byte alignment.
DEC_ALIGN_128B	128-byte alignment.
DEC_ALIGN_256B	256-byte alignment.
DEC_ALIGN_512B	512-byte alignment.
DEC_ALIGN_1024B	1024-byte alignment.
DEC_ALIGN_2048B	2048-byte alignment.

4.1.2.9 DecCodec

enum `DecCodec`

Defines decoder types.

Enumerator

DEC_MPEG4	MPEG-4 decoder.
DEC_MPEG2	MPEG-2 decoder.
DEC_VP6	VP6 decoder.
DEC_VP8	VP8 decoder.
DEC_VP9	VP9 decoder.
DEC_HEVC	HEVC (H.265) decoder.

Enumerator

DEC_H264	H.264 (AVC) decoder.
DEC_AVS	AVS decoder.
DEC_AVS2	AVS2 decoder.
DEC_AV1	AV1 decoder.
DEC_JPEG	JPEG decoder.
DEC_VC1	VC1 decoder.
DEC_RV	RV decoder.
DEC_VVC	VVC decoder.
DEC_PPDEC	Decoder that serves as a standalone post-processor.
DEC_AVS3	Support AVS3 video decoder.
DEC_FMT_MAX	The limit on the input value of this parameter.

4.1.2.10 DecRet

enum **DecRet**

Specifies the return values for the decoder API functions.

Enumerator

DEC_OK	The API call is successful.
DEC_STRM_PROCESSED	Processing of the stream is finished.
DEC_PIC_RDY	The picture is ready for output.
DEC_PIC_DECODED	The picture is decoded.
DEC_HDRS_RDY	New stream headers are decoded.
DEC_DP_HDRS_RDY	(Only for MPEG-4) New stream headers are decoded for data-partitioned streams.
DEC_ADVANCED_TOOLS	Advanced coding tools such as ASO and FMO are detected in the stream.
DEC_SLICE_RDY	A slice is decoded.
DEC_PENDING_FLUSH	Output pictures must be retrieved before continuing decode.
DEC_NONREF_PIC_SKIPPED	Non-reference pictures are skipped.

Enumerator

DEC_END_OF_STREAM	The decoder detects the end-of-stream state.
DEC_END_OF_SEQ	The end of the video sequence is reached.
DEC_WAITING_FOR_BUFFER	The decoder is waiting for external buffer allocation.
DEC_SCAN_PROCESSED	(Only for PJPEG) The scan is processed.
DEC_ABORTED	The decoder is aborted.
DEC_FLUSHED	All pictures have been flushed.
DEC_BUF_EMPTY	The input buffer has been consumed, but the decoding is not completed.
DEC_RESOLUTION_CHANGED	The resolution is changed in the multi-resolution video.
DEC_VOS_END	The end marker of the video object sequence is dedected.
DEC_PARAM_SET_PARSED	The VPS, PPS, or SPS is parsed.
DEC_SEI_PARSED	The SEI is parsed.
DEC_PB_PIC_SKIPPED	Decoding of P- and B-pictures is skipped.
DEC_DISCARD_INTERNAL	The output picture is discarded.
DEC_PARAM_ERROR	Invalid parameters are used.
DEC_STRM_ERROR	An unrecoverable error occurs in decoding.
DEC_NOT_INITIALIZED	The decoder is not initialized.
DEC_MEMFAIL	Memory allocation fails.
DEC_INITFAIL	Initialization fails.
DEC_HDRS_NOT_RDY	Video sequence information is unavailable because stream headers are not decoded.
DEC_STREAM_NOT_SUPPORTED	The video sequence frame size or tools are not supported.
DEC_EXT_BUFFER_REJECTED	External buffer rejected. (Too much than requested)
DEC_INFOPARAM_ERROR	Invalid post-processing parameters are used.
DEC_ERROR	An error occurs.
DEC_UNSUPPORTED	Unsupported.
DEC_INVALID_STREAM_LENGTH	The stream length is invalid.
DEC_INVALID_INPUT_BUFFER_SIZE	The input buffer size is invalid.
DEC_INCREASE_INPUT_BUFFER	Input buffer size is insufficient. Please allocate a larger input buffer.

Enumerator

DEC_SLICE_MODE_UNSUPPORTED	The slice mode is not supported for this operation type.
DEC_METADATA_FAIL	The metadata is in an improper format.
DEC_NO_REFERENCE	The current picture has no reference pictures, and is skipped.
DEC_PARAM_SET_ERROR	An error occurs when parsing the VPS, SPS, or PPS.
DEC_SEI_ERROR	An error occurs when extracting or parsing the SEI.
DEC_SLICE_HDR_ERROR	An error occurs when parsing the slice header.
DEC_REF_PICS_ERROR	An error occurs when setting reference pictures for the current picture.
DEC_NALUNIT_ERROR	An error occurs when extracting or parsing the SEI.
DEC_AU_BOUNDARY_ERROR	An error occurs when checking AU boundaries.
DEC_STREAM_ERROR_DETECTED	An error occurs in syntax elements of the stream.
DEC_NO_DECODING_BUFFER	The decoder has no available buffer.
DEC_HW_RESERVED	The driver cannot reserve decoder hardware.
DEC_HW_TIMEOUT	The decoder hardware times out.
DEC_HW_BUS_ERROR	The decoder hardware receives the error status from the system bus.
DEC_SYSTEM_ERROR	An unrecoverable system error occurs in the decoder hardware.
DEC_DWL_ERROR	An error occurs in the decoder wrapper layer (DWL).
DEC_FATAL_SYSTEM_ERROR	A fatal system error occurs in the hardware.
DEC_HW_EXT_TIMEOUT	An external timeout occurs.
DEC_EVALUATION_LIMIT_EXCEEDED	The evaluation limit is exceeded.
DEC_FORMAT_NOT_SUPPORTED	The video codec standard is not supported.

4.1.2.11 DecDARFormat

```
enum DecDARFormat
```


(Only for AVS and MPEG-2) Defines display aspect ratios (DARs).

Enumerator

DEC_1_1	DAR 1 : 1.
DEC_4_3	DAR 4 : 3.
DEC_16_9	DAR 16 : 9.
DEC_2_21_1	DAR 2.21 : 1.

4.1.2.12 DecInputFormat

```
enum DecInputFormat
```

(Only for VP8 and MPEG-4) Defines input sequence formats.

Enumerator

DEC_INPUT_VP7	The VP7 format.
DEC_INPUT_VP8	The VP8 format.
DEC_INPUT_WEBP	The WebP format.
DEC_INPUT_MPEG4	The MPEG-4 format.
DEC_INPUT_SORENSEN	The Sorenson format.
DEC_INPUT_CUSTOM_1	A custom format.

4.1.2.13 DecSkipFrameMode

```
enum DecSkipFrameMode
```

Defines frame skipping modes for non-reference frames.

Enumerator

DEC_SKIP_NON_REF_RECON	(Default) Decodes all frames and does not output reconstructed data of non-reference frames. This value is available only for H.264 (AVC) of the High 10 profile and for HEVC (H.264).
DEC_SKIP_NON_REF	Does not decode non-reference frames.
DEC_SKIP_NONE	Does not skip non-reference frames during decoding.

4.1.2.14 DecVideoRange

enum `DecVideoRange`

Defines YUV sample ranges for decoded pictures.

NOTE: DEC_VIDEO_RANGE_NORMAL is equivalent to DEC_VIDEO_STUDIO_SWING.
DEC_VIDEO_RANGE_FULL is equivalent to DEC_VIDEO_FULL_SWING.

Enumerator

DEC_VIDEO_RANGE_NORMAL	Limited ranges, where Y samples are in range [16, 235].
DEC_VIDEO_RANGE_FULL	Full ranges, where Y samples are in range [0, 255].
DEC_VIDEO_STUDIO_SWING	Limited ranges, where Y samples are in range [16, 235].
DEC_VIDEO_FULL_SWING	Full ranges, where Y samples are in range [0, 255].

4.1.2.15 DecColorSpace

enum `DecColorSpace`

Defines color spaces for decoded pictures.

Enumerator

DEC_YCbCr_BT601	The YUV color space.
DEC_CUSTOM	A custom color space.
DEC_RGB	The RGB color space.

4.1.2.16 DecBufferType

enum `DecBufferType`

Defines buffer types.

Enumerator

REFERENCE_BUFFER	Reference frame buffer, reference frame compression (RFC) table buffer, or direct motion vector (DMV) buffer.
RASTERSCAN_OUT_BUFFER	raster scan out buffer
DOWNSCALE_OUT_BUFFER	douwnscale scan out buffer

4.1.2.17 DecUserDataType

enum `DecUserDataType`

(Only for MPEG-4) Defines user data types.

Enumerator

DEC_USER_DATA_VOS	The VOS type.
DEC_USER_DATA_VISO	The VISO type.
DEC_USER_DATA_VOL	The VOL type.
DEC_USER_DATA_GOV	The GOV type.

4.1.3 Functions

4.1.3.1 VCDecGetAPIVersion()

```
struct DecApiVersion VCDecGetAPIVersion (
    void )
```

Returns the decoder's API version. This function does not require the creation of a decoder instance.

Parameters

in	void	None.
----	------	-------

Returns

DecApiVersion struct DecApiVersion.

4.1.3.2 VCDecGetBuild()

```
struct DecSwHwBuild VCDecGetBuild (
    u32 client_type )
```

Returns the decoder's hardware and software build information. This function does not require the creation of a decoder instance.

Parameters

in	client_type	Specifies the client type.
----	-------------	----------------------------

Returns

DecSwHwBuild struct DecSwHwBuild.

4.1.3.3 VCDecMCGetCoreCount()

```
u32 VCDecMCGetCoreCount (
    void )
```

This function read number of HW cores available (Multicore specific). This function does not require the creation of a decoder instance.

Parameters

in	void	None.
----	------	-------

Returns

u32 The number of available hardware decoding cores..

4.1.3.4 VCDecUnpackMetaData()

```
enum DecRet VCDecUnpackMetaData (
    enum DecCodec codec,
    const u8 * p_buffer,
    u32 buffer_size,
    struct DecMetaData * p_meta_data )
```

Unpacks metadata elements for sequence header C from buffer, when metadata is packed according to SMPTE VC-1 Standard Annex J(VC1 specific). This function does not require the creation of a decoder instance.

Parameters

in	<i>codec</i>	Specifies the decoder types.
in	<i>p_buffer</i>	Pointer to buffer containing packed metadata. Buffer must contain at least 4 bytes.
in	<i>buffer_size</i>	Buffer size in bytes.
in	<i>p_meta_data</i>	Pointer to stream metadata container.

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR, DEC_METADATA_FAIL.

4.1.3.5 VCDecInit()

```
enum DecRet VCDecInit (
    const void ** inst,
    struct DecInitConfig * init_config )
```

Initializes decoder software. Function reserves memory for the decoder instance and calls xxxInit to initialize the instance data.

Parameters

in	<i>inst</i>	Pointer to the location where this function returns a decoder instance. This instance is later passed to other decoder API functions.
in	<i>dec_cfg</i>	Pointer to the decoder initialization parameters.

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR, DEC_MEMFAIL, DEC_FORMAT_NOT_SUPPORTED.

4.1.3.6 VCDecGetInfo()

```
enum DecRet VCDecGetInfo (
    void * inst,
    struct DecSequenceInfo * info )
```

This function provides read access to decoder information. This function should not be called before VCDecDecode function has indicated that headers are ready.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>info</i>	Pointer to an DecSequenceInfo structure where the decoder information will be returned.

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR, DEC_HDRS_NOT_RDY,
DEC_NOT_INITIALIZED

4.1.3.7 VCDecSetInfo()

```
enum DecRet VCDecSetInfo (  
    void * inst,  
    struct DecConfig * config )
```

This function is called by the client to set the decoder configuration.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>config</i>	Pointer to the decoder configure parameters (DecConfig).

Returns

DecRet value: DEC_OK, DEC_NOT_INITIALIZED, DEC_PARAM_ERROR

4.1.3.8 VCDecDecode()

```
enum DecRet VCDecDecode (  
    void * inst,
```

```
struct DecOutput * output,  
struct DecInputParameters * param )
```

This function decodes one or more NAL units from the current stream.



Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>output</i>	Pointer to the decoder output parameters.
in	<i>param</i>	Pointer to the decoder input parameters.

Returns

DecRet value: DEC_PIC_DECODED, DEC_HDRS_RDY, DEC_PARAM_ERROR, DEC_STRM_ERROR, DEC_NOT_INITIALIZED, DEC_HW_BUS_ERROR, DEC_HW_TIMEOUT, DEC_MEMFAIL, DEC_STREAM_NOT_SUPPORTED, DEC_NONREF_PIC_SKIPPED, DEC_WAITING_FOR_BUFFER, DEC_ABORTED, DEC_STRM_PROCESSED, DEC_SYSTEM_ERROR, DEC_HW_RESERVED, DEC_PENDING_FLUSH, DEC_NO_DECODING_BUFFER, DEC_BUF_EMPTY

4.1.3.9 VCDecNextPicture()

```
enum DecRet VCDecNextPicture (
    void * inst,
    struct DecPictures * pic )
```

Provides access to the next picture in display order.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>pic</i>	Pointer to a DecPictures structure which used to return the picture parameters. The picture parameters are valid only when the return value indicates that an output picture is available.

Returns

DecRet value: DEC_OK, DEC_PIC_RDY, DEC_PARAM_ERROR, DEC_NOT_INITIALIZED, DEC_END_OF_STREAM, DEC_ABORTED, DEC_FLUSHED

4.1.3.10 VCDecPictureConsumed()

```
enum DecRet VCDecPictureConsumed (
    void * inst,
    struct DecPictures * pic )
```

This function informs the decoder that the client has finished processing the specific picture and releases the picture buffer for the decoder to process the next picture.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>pic</i>	Pointer to a DecPictures structure which used to return the picture parameters. The picture parameters are valid only when the return value indicates that an output picture is available.

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR, DEC_NOT_INITIALIZED

4.1.3.11 VCDecEndOfStream()

```
enum DecRet VCDecEndOfStream (
    void * inst )
```

This function informs the decoder that it should not be expecting any more input stream and to finish decoding and outputting all the buffers that are currently pending in the component.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
----	-------------	--

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR, DEC_INIT_FAIL

4.1.3.12 VCDecGetBufferInfo()

```
enum DecRet VCDecGetBufferInfo (
    void * inst,
    struct DecBufferInfo * buf_info )
```

This function is called by the client to get the frame buffer information requested by the decoder, including the requested buffer size and buffer numbers.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>buf_info</i>	Pointer to a DecBufferInfo structure which used to return the buffer info.

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR, DEC_WAITING_FOR_BUFFER

4.1.3.13 VCDecAddBuffer()

```
enum DecRet VCDecAddBuffer (
    void * inst,
    struct DWLLinearMem * buf )
```

This function is called by the client to add an external frame buffer into the decoder.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>buf</i>	Pointer to a DWLLinearMem structure.

Returns

DecRet value: DEC_PARAM_ERROR, DEC_EXT_BUFFER_REJECTED, DEC_WAITING_FOR_BUFFER

4.1.3.14 VCDecUseExtraFrmBuffers()

```
enum DecRet VCDecUseExtraFrmBuffers (
    void * inst,
    u32 n )
```

This function is called by the client to indicates that 'n' extra frames in DPB are needed.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>n</i>	Need 'n' extra frames in DPB.

Returns

DecRet value: DEC_OK.

4.1.3.15 VCDecPeek()

```
enum DecRet VCDecPeek (
    void * inst,
    struct DecPictures * pic )
```

Get last decoded picture if any available. No pictures are removed from output nor DPB buffers.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>pic</i>	Pointer to a DecPictures structure which used to return the picture parameters. The picture parameters are valid only when the return value indicates that an output picture is available.

Returns

DecRet value: DEC_OK, DEC_PIC_RDY, DEC_PARAM_ERROR, DEC_NOT_INITIALIZED

4.1.3.16 VCDecAbort()

```
enum DecRet VCDecAbort (
    void * inst )
```

This function is called by the client to abort the decoder.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
----	-------------	--

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR, DEC_NOT_INITIALIZED

4.1.3.17 VCDecAbortAfter()

```
enum DecRet VCDecAbortAfter (
    void * inst )
```

This function is called by the client to reset the decoder after the decoder is aborted.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
----	-------------	--

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR, DEC_NOT_INITIALIZED

4.1.3.18 VCDecSetNoReorder()

```
void VCDecSetNoReorder (
    void * inst,
    u32 no_reorder )
```

This function is called by client to set the decoder output picture order as either decoding order or display order.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>no_reorder</i>	Set the output model to no_reordering.

Returns

void None.

4.1.3.19 VCDecRelease()

```
void VCDecRelease (
    void * inst )
```

Releases the decoder instance. Function used to release instance data and frees the memory allocated for the instance.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit. The decoder instance to be released.
----	-------------	---

Returns

void None.

4.1.3.20 VCDecUpdateStrmInfoCtrl()

```
void VCDecUpdateStrmInfoCtrl (
    void * inst,
    struct strmInfo info )
```

This function is related to low latency mode and is called by the client to update the decoder stream length register at the specified address.

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>info</i>	Stream information data structure.

Returns

void None.

4.1.3.21 VCDecGetUserData()

```
enum DecRet VCDecGetUserData (
    void * inst,
    struct DecInputParameters * param,
    struct DecUserConf * user_data_config )
```

This function is used to get user data information (mpeg4 specific).

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>param</i>	Pointer to the input parameters.
in	<i>user_data_config</i>	Pointer to the user data configuration.

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR.

4.1.3.22 VCDecSetCustomInfo()

```
enum DecRet VCDecSetCustomInfo (  
    void * inst,  
    const u32 width,  
    const u32 height )
```

Provide external information to decoder. Used for some custom streams which do not contain all necessary information in the elementary bitstream. (mpeg4 specific).

Parameters

in	<i>inst</i>	A decoder instance created earlier with a call to VCDecInit.
in	<i>width</i>	Frame width in pixels.
in	<i>height</i>	Frame height in pixels.

Returns

DecRet value: DEC_OK, DEC_PARAM_ERROR.

4.1.3.23 VCDecRetStr()

```
char * VCDecRetStr (  
    enum DecRet rv )
```

Retrieve the string of decoder return code.

Parameters

in	<i>DecRet</i>	Decoder return code.
----	---------------	----------------------

Returns

char * Pointer to human readable string of return code.



5 Data Structure Documentation

5.1 DecApiVersion

Get API version.

Data Fields

- **u32 major**
API major version.
- **u32 minor**
API minor version.
- **u32 micro**
API micro version.

5.2 DecBufferInfo

A structure containing the reference buffer information.

Data Fields

- **u64 next_buf_size**
- **u32 buf_num**
extra buffer number
- **u32 add_extra_ext_buf**
number of extra buffer has been added
- **struct DWLLinearMem buf_to_free**
the initialized buffer
- **u32 ystride** [DEC_MAX_OUT_COUNT]
- **u32 cstride** [DEC_MAX_OUT_COUNT]

5.2.1 Field Documentation

5.2.1.1 next_buf_size

u64 next_buf_size

extra buffer size

5.3 DecConfig

Decoder setting params, will be used in VCDecSetInfo.

Data Fields

- **PpUnitConfig ppu_cfg [DEC_MAX_OUT_COUNT]**
A structure containing PP unit configuration information.
- **DelogoConfig delogo_params [2]**
delogo config parameters
- **DecPicAlignment align**
Stride alignment: aligned to 1/8/16/.../512 bytes.
- **u32 hw_conceal**
Use Hw error concealment(only hevc/h264/av1).
- **u32 disable_slice**
disable slice mode(only hevc/h264).
- **i8 max_temporal_layer**
The maximum temporal layer that the VCD supports decoding(only hevc/h264/vvc).
- **u32 dec_image_type**
decode image type : JPEGDEC_IMAGE or JPEGDEC_THUMBNAIL
- **u32 chroma_format**
decode chroma format : 420,422 etc
- **u32 in_format**
The format of input.
- **u32 in_stride**

- pic stride of the input.*
- **u32 in_height**
pixels height of the input.
- **u32 in_width**
pixels width of the input.
- **u32 in_lu_stride**
input stride of RFC Luma data.
- **u32 in_ch_stride**
input stride of RFC Chroma data.
- **u32 in_lut_stride**
input stride of RFC Luma table data.
- **u32 in_cht_stride**
input stride of RFC Chroma table data.
- **u32 in_luma_bitdepth**
input bitdepth of RFC Luma data.
- **u32 in_chroma_bitdepth**
input bitdepth of RFC Chroma data.
- **u8 in_rfc**
flag indicating the input buffer is the compressed output from RFC.
- struct DWLLinearMem **pp_in_buffer**
The buffer malloced for pp input.
- struct DWLLinearMem **pp_out_buffer**
The buffer malloced for pp output.
- **u32 crc_enabled**
Flag indicating whether checksum enabled. (must used with vcmd)

5.4 DecCropParams

Cropping info in samples describing a rectangular region specified in picture coordinates for output.

Data Fields

- **u32 crop_left_offset**
The left offset in samples.
- **u32 crop_out_width**

The width in samples.

- **u32 crop_top_offset**

The top offset in samples.

- **u32 crop_out_height**

The height in samples.

5.5 DeclInitConfig

Decoder initialization params, will be used in VCDeclInit.

Data Fields

- **const void * dwl_inst**
struct DWL instance.
- **enum DecCodec codec**
Specifies the decoder types.
- **u32 disable_picture_reordering**
When set a non-zero value, the decoded pictures will be output in decoding order without reordering.
- **enum DecErrorHandling error_handling**
Error handling mode.
- **u32 error_ratio**
*Error ratio percentage multiplied by 100. Valid range is [0, 100]. A picture with erroneous blocks percentage * 100 <= error_ratio will be treated as a "good" picture. It's used when error handling strategy is DEC_EC_OUT_DECISION.*
- **u32 use_video_compressor**
Flag indicating whether RFC is to be enabled.
- **u32 multi_frame_input_flag**
Specifies the input stream of decoder contains at least two frames.
- **enum DecDecoderMode decoder_mode**
Decoder working mode.
- **u32 guard_size**
The minimum difference between minimum buffers number and allocated buffers number that will force to return HDRS_RDY even buffers number/size are sufficient for new sequence.
- **u32 use_adaptive_buffers**
When sequence changes, if old output buffers (number/size) are sufficient for new sequence, old buffers will be used instead of reallocating output buffer.
- **u32 support_asofmo_stream**
Indicates whether aso/fmo stream is supported.

- **u32 num_frame_buffers**

To use *n* frame buffers in decoder specified by user.

- **u32 rlc_mode**

RLC is a special decoding mode for H264/MPEG4 only. When RLC mode is enabled, SW will perform the bitstream entropy decoding other than HW and store the intermediate data in external rlc buffer. HW will read from this external buffer and continue following decoding stages. The flag indicates whether RLC mode is enabled.

- **struct DecMCConfig mc_cfg**

Multicore decoder init configuration.

- **enum DecSkipFrameMode skip_frame**

Skip some frames when decoding.

- **u32 auxinfo**

Flag to enable aux info output. 0: disable aux info dumping, 1: enable QP dumping, 2: enable MV dumping, 3: enable both QP and MV dumping.

- **enum DecInputFormat dec_format**

Format of the input sequence (vp8/mpeg4 only).

- **u32 mvc**

Flag indicating whether it's a MVC (Multi-View Coding) stream: 1 for MVC stream, otherwise 0 for non-MVC stream.

- **u32 frame_code_length**

Frame code length in bits, which informs how many bits in stream are used for frame_size_code syntax (HW discards these bits)

- **u32 * frame_sizes**

Array of dimensions of possible resampled images sizes. For each frame, first 4 bytes are width following by next 4 bytes of height.

- **u32 rv_version**

RV version: 0 for RV8, otherwise 1 (RV9/RV10)

- **u32 max_frame_width**

Max frame width for coded pictures in the stream in pixels.

- **u32 max_frame_height**

Max frame height for coded pictures in the stream in pixels.

- **struct DecMetaData meta_data**

The metadata of the stream (Only used in VC1)

- **u32 annexb**

Flag indicating the stream to be decoded conforms to "Annex B: Length delimited bitstream format" defined in AV1 spec.

- **u32 plainobu**

Flag indicating input stream are packetized in "Open Bitstream Units" or OBUs. Each OBU has a header, which provides identifying information for the contained data (payload).

- **u32 multicore_poll_period**

Flag indicating tiles to be decoded will be in tile column first order, that is, all the tiles in the first column will be decoded from above to bottom, then the second tile column ...

- **u32 tile_transpose**

- **u32 oppoints**

Flag indicating whether all the layers are included in operating point. An operating point specifies which spatial and temporal layers should be decoded. 1 indicates all the layers are to be decoded, 0 means only to decode layer 0.

- **u32 crc_enabled**

Flag indicating whether checksum enabled. (must used with vcmd)

5.5.1 Field Documentation

5.5.1.1 multicore_poll_period

u32 multicore_poll_period

Polling period with multicore decoding. Please refer to sw_dec_mc_polltime in register description.

5.6 DecInputParameters

Decoder input data.

Data Fields

- **u32 pic_id**

picture decoded id number

- **void * p_user_data**

point to user data

- **u8 * stream**

input stream virtual address

- **u32 strm_len**

input stream length

- **addr_t stream_bus_address**

input stream bus address

- **enum DecSkipFrameMode skip_frame**

Skip some frames when decoding.

- struct DWLLinearMem **stream_buffer**

input stream buffer

- struct **SEI_buffer** * **sei_buffer**

SEI buffer, HEVC and H264 apecific.

- u32 **low_latency**

Indicates whether low latency is available.

- u32 **pic_decoded**

flag : this pic has been decoded in low latency mode.

- u32 **exit_send_thread**

flag : exit the data send thread.

- u32 **frame_len**

The one frame data length in low latency mode.

- u8 * **strm_vir_start_addr**

input stream virtual address

- u32 **enable_deblock**

Enable deblocking of post processed picture. NOTE: This parameter is not valid if the decoder is not used in pipeline mode with the post processor i.e. it has no effect on the decoding process.

- u32 **slice_height**

height of WebP slice, unused for other formats

- u32 * **p_pic_buffer_y**

luminance output address of user allocated buffer, used in conjunction with external buffer allocation

- addr_t **pic_buffer_bus_address_y**

The bus address for luminance output.

- u32 * **p_pic_buffer_c**

chrominance output address of user allocated buffer, used in conjunction with external buffer allocation

- addr_t **pic_buffer_bus_address_c**

The bus address for luminance output.

- u32 **dec_image_type**

Full image or Thumbnail to be decoded.

- u32 **slice_mb_set**

slice mode: mcu rows to decode

- u32 **ri_count**

count of restart intervals in input stream

- u32 * **ri_array**

offsets of beginnings of restart intervals

- u32 **buffer_size**

input stream buffer size in input stream buffering mode

- struct DWLLinearMem **picture_buffer_y**
luma output address ==> if user allocated
- struct DWLLinearMem **picture_buffer_cb_cr**
chroma output address ==> if user allocated
- struct DWLLinearMem **picture_buffer_cr**
chroma output address ==> if user allocated
- u32 **timestamp**
timestamp of current picture from rv frame header. NOTE: timestamp of a B-frame should be adjusted referring to its forward reference frame's timestamp
- u32 **slice_info_num**
The number of slice offset entries.
- struct **DecSliceInfo** * **slice_info**
Pointer to the slice_info.

5.7 DecMCConfig

Multicore decoder init configuration.

Data Fields

- u32 **mc_enable**
Indicates whether multicore is available.
- **DecMCStreamConsumed** * **stream_consumed_callback**
Application provided callback for stream buffer processed.

5.8 DecMetaData

Container for the metadata of the stream(Only used in VC1).

Contains a wide range of information about the stream, e.g. what kind of tools are needed to decode the stream.

Data Fields

- **u32 max_coded_width**
Specifies the maximum coded width in pixels of picture within the sequence. Valid range [2,8192] (even values).
- **u32 max_coded_height**
Specifies the maximum coded height in pixels of picture within the sequence. Valid range [2,8192] (even values).
- **u32 vs_transform**
Indicates whether variable sized transform is enabled for the sequence. Valid range [0,1].
- **u32 overlap**
Indicates whether overlap smoothing is enabled for the sequence. Valid range [0,1].
- **u32 sync_marker**
Indicates whether there are synchronization markers in the stream. Valid range [0,1].
- **u32 quantizer**
Indicates quantizer type used for the sequence. Valid range [0,3].
- **u32 frame_interp**
Indicates whether the INTERPFRM flag (which provides information to display process) exists in the picture headers. Valid range [0,1].
- **u32 max_bframes**
Specifies the maximum amount of consecutive B-frames within the sequence. Valid range [0,7].
- **u32 fast_uv_mc**
Indicates whether the rounding of color difference motion vectors is enabled. Valid range [0,1].
- **u32 extended_mv**
Indicates whether extended motion vectors are enabled for the sequence. Valid range [0,1].
- **u32 multi_res**
Indicates whether frames may be coded at smaller resolutions than the specified frame resolution. Valid range [0,1].
- **u32 range_red**
Indicates whether range reduction is used in the sequence. Valid range [0,1].
- **u32 dquant**
Indicates whether the quantization step may vary within a frame. Valid range [0,2].
- **u32 loop_filter**
Indicates whether loop filtering is enabled for the sequence. Valid range [0,1].
- **enum VC1Profile profile**
Specifies profile of the input video bitstream.

5.9 DecOutput

The decoder output.

Data Fields

- **u8 * strm_curr_pos**
Pointer to stream position where decoding ended.
- **addr_t strm_curr_bus_address**
The bus address location where the decoding ended.
- **u32 data_left**
How many bytes left undecoded.
- **u8 * strm_buff**
Pointer to stream buffer start.
- **addr_t strm_buff_bus_address**
The bus address location where the stream buffer start.
- **u32 buff_size**
The stream buffer size.
- **struct SEI_buffer * sei_buffer**
SEI buffer.
- **u32 slice_height**
height of WebP slice, unused for other formats
- **addr_t llstrm_curr_address**

5.9.1 Field Documentation

5.9.1.1 llstrm_curr_address

addr_t llstrm_curr_address

Low latency check the addr for current stream

5.10 DecParams

partial tb_cfg.dec_params parameters..

Data Fields

- **u32 bus_burst_length**
how long are the hardware data bursts; better left unchanged.
- **u32 clk_gate_decoder**
clock is gated from decoder between images and for disabled codecs.
- **u32 non_seq_clk**
reference buffer : memory wait states for non_seq_clk.
- **u32 seq_clk**
reference buffer : memory wait states for seq_clk.
- **i32 apf_threshold_value**
APF threshold value.
- **u32 axi_wr_outstand**
AXI outstanding threshold for write.
- **u32 axi_rd_outstand**
AXI outstanding threshold for read.

5.11 DecPicture

Structure to carry information about decoded pictures.

Data Fields

- struct **DecSequenceInfo** **sequence_info**
Sequence coding parameters used.
- struct DWLLinearMem **luma**
Buffer properties.
- struct DWLLinearMem **chroma**
Buffer properties : cb.
- struct DWLLinearMem **chroma_cr**
Buffer properties : cr.
- struct DWLLinearMem **dmv**
DMV Buffer properties.
- struct DWLLinearMem **qp**
QP Buffer properties.
- struct DWLLinearMem **luma_table**

Buffer properties for luma tile status address of dec or rfc.

- struct DWLLinearMem **chroma_table**

Buffer properties for chroma tile status address of dec or rfc.

- struct **DecPictureInfo picture_info**

Picture specific parameters.

- struct DecSEIParameters **sei_param**

SEI parameters of all formats.

- struct DWLLinearMem **hdr**

- u32 **pic_width**

picture width

- u32 **pic_height**

picture height

- u32 **pic_stride**

picture stride for luma

- u32 **pic_stride_ch**

picture stride for chroma

- u32 **chroma_format**

- u32 **pp_enabled**

post process is available

- u32 **is_gdr_frame**

Indicates if picture is a GDR picture.

- u32 * **crc**

Store the results of checksum of output data.

- u32 **top_field_first**

1-top field first; 0-bottom field first

- u32 **fields_in_picture**

how many fields in picture:0-no field;1-one field;2-two fields

5.12 DecPictureInfo

Picture specific information.

Data Fields

- enum **DecPicCodingType** **pic_coding_type**
Picture coding type.
- enum **DecPicCodingType** **pic_coding_type_field**
Picture coding type for bottom field.
- u32 **is_corrupted**
Tells whether picture is corrupted.
- enum **DecPictureFormat** **format**
Color format of the picture.
- struct **DecTime** **time_code**
Time info(Only used in AVS, MP2 and MP4).
- u32 **cycles_per_mb**
Average decoding time in cycles per mb.
- u32 **pic_id**
Identifier for the picture to be decoded.
- u32 **decode_id**
Identifier for the picture to be decoded.
- i32 **poc**
- u32 **is_intra_frame**
Indicates if Frame is an Intra Frame.
- u32 **is_intra_frame_field**
Indicates if Frame is an Intra Frame for bottom field.
- u32 **is_golden_frame**
Indicates if Frame is a Golden reference Frame.
- u32 **field_picture**
Indicates if Frame is a field Frame.
- u32 **top_field**
Indicates if this is a top field.
- u32 **first_field**
Indicates if this is a first field.
- u32 **repeat_first_field**
- u32 **single_field**
- u32 **output_other_field**
- u32 **repeat_frame_count**
- u32 **nbr_of_err_mbs**
Number of concealed macroblocks in the frame

- **u32 num_slice_rows**
Number of luminance pixels rows in WebP output picture buffer. If set to 0, whole picture ready.
- **u32 last_slice**
last slice flag
- **u32 luma_stride**
pixel row stride for luminance
- **u32 chroma_stride**
pixel row stride for chrominance
- **u32 view_id**
Identifies the view to which the output picture belongs.

5.13 DecPictures

Structure to carry information about decoded pictures for 'n' pp channel.

Data Fields

- struct **DecPicture** **pictures** [**DEC_MAX_OUT_COUNT**]

5.14 DecSequenceInfo

Video sequence information.

Data Fields

- **u32 vp_version**
VP codec version defined in input stream.
- **u32 vp_profile**
VP coccec profile defined in input stream.
- **u32 pic_width**
decoded picture width in pixels
- **u32 pic_height**
decoded picture height in pixels

- **u32 scaled_width**
scaled width of the displayed video
- **u32 scaled_height**
scaled height of the displayed video
- **u32 pic_width_thumb**
decoded picture width in pixels in Thumbnail
- **u32 pic_height_thumb**
decoded picture height in pixels in Thumbnail
- **u32 scaled_width_thumb**
scaled width of the displayed video in Thumbnail
- **u32 scaled_height_thumb**
scaled height of the displayed video in Thumbnail
- **u32 sar_width**
sample aspect ratio
- **u32 sar_height**
sample aspect ratio
- **u32 dis_comformance_window**
comformance window disabled
- **struct DecCropParams crop_params**
Cropping parameters for the picture.
- **enum DecVideoRange video_range**
YUV sample video range.
- **u32 matrix_coefficients**
matrix coefficients RGB->YUV conversion
- **u32 is_mono_chrome**
is sequence monochrome
- **u32 is_interlaced**
is sequence interlaced
- **u32 num_of_ref_frames**
Maximum number of reference frames.
- **u32 chroma_format_idc**
0 mono chrome; 1 yuv420; 2 yuv422
- **u32 bit_depth_luma**
Bit depth of stored picture for luma.
- **u32 bit_depth_chroma**
Bit depth of stored picture for chroma.
- **u32 out_bit_depth**
Bit depth of output picture(avs2 special)

- enum **DecDpbMode** **dpb_mode**
DPB mode : frame, or field interlaced.
- enum **DecPictureFormat** **output_format**
Format of the output picture.
- enum **DecPictureFormat** **output_format_thumb**
Format of the output picture in thumbnail.
- enum **JpegCodingMode** **coding_mode**
JPEG code type : JPEG_BASELINE JPEG_PROGRESSIVE JPEG_NONINTERLEAVED.
- enum **JpegCodingMode** **coding_mode_thumb**
JPEG code type in thumbnail : JPEG_BASELINE JPEG_PROGRESSIVE JPEG_NONINTERLEAVED.
- u32 **thumbnail_type**
Thumbnail exist or not or not supported.
- u32 **pp_enabled**
post process is available
- u32 **h264_base_mode**
baseline profile, only H264
- u32 **frame_rate_numerator**
- u32 **frame_rate_denominator**
- u32 **buf_release_flag**
- struct **DecInputParameters** **jpeg_input_info**
Decoder input data(Only jpeg in here)
- u32 **img_max_dec_width**
Maximum supported decoding width for jpeg.
- u32 **img_max_dec_height**
Maximum supported decoding height for jpeg.
- u32 **profile_id**
The profile id.
- u32 **level_id**
The level id.
- u32 **profile_and_level_indication**
- u32 **display_aspect_ratio**
- u32 **stream_format**
- u32 **video_format**
- u32 **colour primaries**
- u32 **transfer_characteristics**
- u32 **colour_description_present_flag**
- u32 **trc_status**
0-both filled 1-only filled transfer_characteristics 2-only filled preferred_transfer_characteristics

- u32 **preferred_transfer_characteristics**
- u32 **multi_buff_pp_size**
- u32 **user_data_voslen**

length of VOS data

- u32 **user_data_visolen**

length of VIS0 data

- u32 **user_data_vollen**

length of VOL data

- u32 **user_data_govlen**

length of GOV data

- u32 **gmc_support**

- u32 **out_pic_coded_width**

output picture width

- u32 **out_pic_coded_height**

output picture height

- u32 **out_pic_stat**

- u32 **timing_info_present_flag**

- u32 **num_units_in_tick**

- u32 **time_scale**

- u32 **thumbnail_done**

5.15 DecSliceInfo

Pointer to the slice info(Only used in RV).

It contains offset value of each slice in the data buffer,.

Data Fields

- u32 **offset**

The offset value of each slice in the data buffer, including start point "0" and end point "data_len".

- u32 **is_valid**

Indicates whether slice info is available.

5.16 DecSwHwBuild

Sw and Hw Build information.

Data Fields

- **u32 sw_build**
Software build ID.
- **u32 asic_id**
Hardware aisc ID.
- **u32 hw_build_id** [MAX_ASIC_CORES]
Hardware build ID.

5.17 DecTime

Time info(Only used in AVS, MP2 and MP4).

Data Fields

- **u32 hours**
hours information
- **u32 minutes**
minutes information
- **u32 seconds**
seconds information
- **u32 pictures**
pictures information
- **u32 time_incr**
vop time increment : mpeg4 specific
- **u32 time_res**
vop time increment resolution : mpeg4 specific

5.18 DecUserConf

User data configuration(Only used in MPEG4).

Data Fields

- enum **DecUserDataType** **user_data_type**

The type of buffer allocated.

- **u8 * p_user_data_vos**

The pointer to VOS data.

- **u32 user_data_vosmax_len**

The VOS data max length.

- **u8 * p_user_data_viso**

The pointer to VISO data.

- **u32 user_data_visomax_len**

The VISO data max length.

- **u8 * p_user_data_vol**

The pointer to VOL data.

- **u32 user_data_volmax_len**

The VOL data max length.

- **u8 * p_user_data_gov**

The pointer to GOV data.

- **u32 user_data_govmax_len**

The GOV data max length.

5.19 DelogoConfig

Contains delogo filter configurations.

Data Fields

- **u32 enabled**

Indicates whether delogo filter is available.

- **u32 x**

Coordinates : x.

- **u32 y**

Coordinates : y.

- **u32 w**

Size : width.

- **u32 h**

- Size : height.
- u32 **show**
delogo filter level
- enum **DelogoMode mode**
delogo filter mode
- u32 **Y**
YUV : Y.
- u32 **U**
YUV : U.
- u32 **V**
YUV : V.

5.20 ErrorParams

partial tb_cfg.tb_params parameters for randome error..

Data Fields

- u32 **seed**
random seed.
- u8 **truncate_stream**
truncate the stream by randomize the stream length.
- char **truncate_stream_odds** [24]
truncate odds.
- u8 **swap_bits_in_stream**
swap the stream by random swap the stream bits.
- char **swap_bit_odds** [24]
swap odds.
- u8 **lose_packets**
discard the stream by random discard the packets.
- char **packet_loss_odds** [24]
discard odds.
- u32 **random_error_enabled**
enable or disable random error function.

5.21 PpUnitConfig

A structure containing PP unit configuration information.

Data Fields

- u32 **enabled**
PP unit enabled.
- u32 **tiled_e**
PP unit tiled4x4 output enabled.
- u32 **rgb**
RGB output enabled.
- u32 **rgb_planar**
RGB output planar output enabled.
- u32 **cr_first**
CrCb instead of CbCr.
- u32 **shaper_enabled**
Hardware shaper feature enabled.
- u32 **shaper_no_pad**
shaper no need to pad for frame right edge
- u32 **dec400_enabled**
sw control shaper and dec400
- u32 **planar**
Planar output.
- **DecPicAlignment align**
pp output alignment
- **DecPicAlignment align_h**
pp output height alignment
- u32 **ystride**
Stride for Y plane : If set to 0, software should use the stride calculated from software. When not 0, software should check the validation of the value.
- u32 **cstride**
Stride for C plane : If set to 0, software should use the stride calculated from software. When not 0, software should check the validation of the value.

- struct {
 - u32 **enabled**
whether scaling is enabled
 - u32 **scale_by_ratio**
scaling output size will be calculated by ratio
 - u32 **ratio_x**
indicate flexible mode, or 1/2/4/8 indicate ratio
 - u32 **ratio_y**
indicate flexible mode, or 1/2/4/8 indicate ratio
 - u32 **width**
scaled output width
 - u32 **height**
scaled output height
- } **scale**

scale parameters.

- struct {
 - u32 **enabled**
whether cropping is enabled
 - u32 **set_by_user**
cropping set by user, use this variable to record whether user set crop.
 - u32 **x**
cropping start x
 - u32 **y**
cropping start y
 - u32 **width**
cropping width
 - u32 **height**
cropping height
- } **crop**

crop parameters.

- struct {
 - u32 **enabled**
 - u32 **x**
cropping start x
 - u32 **y**
cropping start y
 - u32 **width**

```

        cropping width
    u32 height
        cropping height
    } crop2

```

crop2 parameters.

- **u32 monochrome**

PP output monochrome (luma only) for YUV output.

- **u32 out_p010**

P010 output with the MSB bits valid.

- **u32 out_1010**

Each channel(Y,U,V) use 10bits.

- **u32 out_I010**

P010 output with the low 10bits valid.

- **u32 out_L010**

L010 output with the LSB bits valid.

- **u32 out_p012**

P012 output with the MSB bits valid.

- **u32 out_I012**

Each channel(Y,U,V) use 12bits.

- **u32 out_be**

absolute

- **u32 out_cut_8bits**

Force 8 bit output.

- **u32 out_yuyv**

422 output packed YUYV

- **u32 out_uyvy**

422 output packed UYVY

- **u32 video_range**

1 - full range, 0 - limited range

- **u32 range_max**

limited max to video range

- **u32 range_min**

limited min to video range

- **u32 rgb_format**

RGB output format: RGB888/BGR888/R16G16B16/...

- **u32 rgb_stan**

color conversion standard applied to set coeffs

- **u32 yuv_source_range**
when there is no stream range information, set the input range
- **u32 set_source_enable**
whether set source range is enabled
- **u32 yuv_target_range**
target range for range mapping
- **u32 set_target_enable**
whether set target range is enabled
- **u32 dither_enable**
whether dither is enabled
- **u32 rgb_alpha**
alpha channel value for RGB format
- **u32 pp_filter**
selector for pp scale algorithm
- **u32 x_filter_param**
when select LANCZOS scale, set the horizontal window param
- **u32 y_filter_param**
when select LANCZOS scale, set the vertical window param
- **u32 tile_mode**
when pp tile enable, set the tiled format
- **u32 pp_comp**
Enable TILED16x16 or TILED32x8 compression in PP.
- **u32 src_sel_mode**
set the center point select algorithm for pp scale, default is DOWN ROUND
- **u32 pad_sel**
whether pad value is used for pp scale for the picture boundary
- **u32 pad_Y**
the luma padding value
- **u32 pad_U**
the Cb padding value
- **u32 pad_V**
the Cr padding value
- **u32 chroma_format**
the output YCbCr format
- **u32 lc_stripe**
number of lines to trigger line counter interrupt
- **u32 luma_offset**
begin offset for luma data write in pp output buffer(heif special)

- **u32 chroma_offset**
begin offset for chroma data write in pp output buffer (heif special)
- **u32 buf_off_set_by_user**
the tag of set luma and chroma offset in pp output buffer by TB (heif special)
- **u32 antialias**
Fixed bicubic or bilinear filter size.

5.21.1 Field Documentation

5.21.1.1 enabled

u32 enabled

whether cropping is enabled

whether scaling is enabled

5.21.1.2 width

u32 width

cropping width

5.21.1.3 height

u32 height

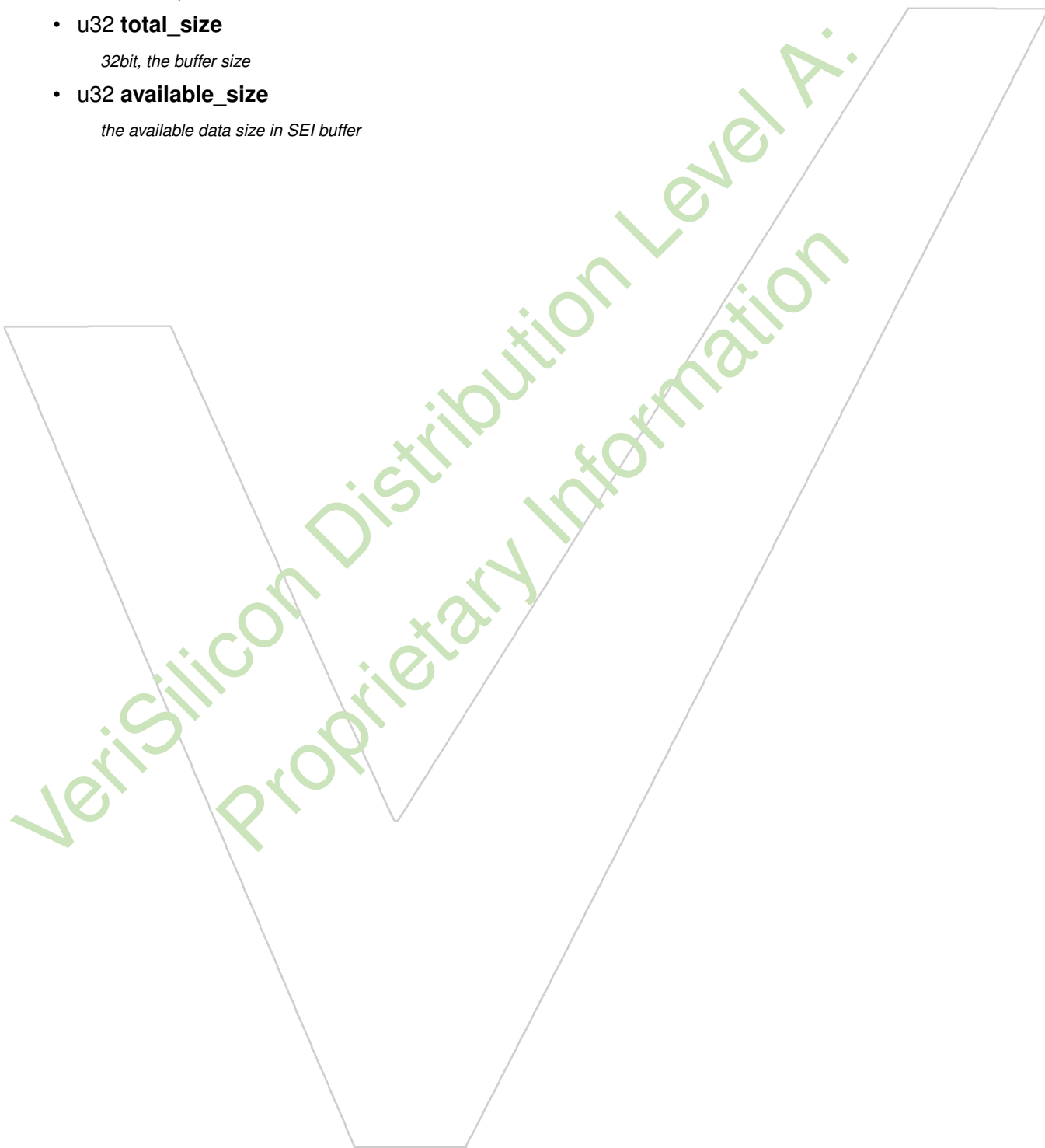
cropping height

5.22 SEI_buffer

It stores captured SEI stream data.

Data Fields

- **u8 bitmask [32]**
256bit, Indicata which SEI is interested
- **u8 * buffer**
SEI buffer pointer.
- **u32 total_size**
32bit, the buffer size
- **u32 available_size**
the available data size in SEI buffer



Document Revision History

This section describes top level differences in the versions of this document.

Note

This document is not necessarily updated for each patch or minor revision. The information in this document tends to be stable across a revision (nnn) series.

Document Revision	Date	Compatible Core	Description
1.03	2024-01-02	VC9000D	Updated the document name and template.
1.02	2022-11-21	VC9000D	- Added components of low latency decoding including the principle and purpose of this design. - Added components of post processor including the enumerations structures and functions.
1.01	2022-10-28	VC9000D	Initial release.